STI v2.0

LE MANUEL CONNECTÉ

LYCÉE RAFÈHA

4ème SI

STI

Sciences et technologies de l’informatique

HTML5 / CSS3 / JAVASCRIPT

BASES DE DONNÉES / PHP

Cours • Méthodes • Travaux pratiques

69 fiches reliées à leur contenu en ligne

Enseignant

Aymen Essouyah

SMART BOOK

Lire. Scanner. Pratiquer.

SCANNER POUR APPRENDRE

ÉDITION CONNECTÉE · 2026

aymenessouyah.github.io/STiV2.0/

Sommaire

MODE D’EMPLOI DU LIVRE CONNECTÉ

Sur écran : cliquez sur le titre du sommaire ou sur « Ouvrir sur STI v2.0 ». Sur papier : scannez le QR contrasté au bas de la fiche pour ouvrir la partie correspondante. Une connexion Internet est nécessaire.

Depuis une photo enregistrée : utilisez la détection de QR de votre téléphone. La reconnaissance automatique du texte d’une page, sans QR lisible, n’est pas intégrée. Le filigrane pâle est une signature discrète.

HTML5

01. Structure d’un document HTML5 ··· 6

02. Syntaxe, attributs et imbrication ··· 7

03. Texte, liens et contenus multimédias ··· 8

04. Listes, tableaux et structure sémantique ··· 9

05. Formulaires : saisie et transmission ··· 10

06. Formulaires : options et aide-mémoire ··· 11

07. Cadres intégrés et cibles de navigation ··· 12

08. Attributs globaux, datalist et événements HTML ··· 13

CSS3

09. Syntaxe et intégration des styles ··· 14

10. Sélecteurs, classes et identifiants ··· 15

11. Unités et modèle de boîte ··· 16

12. Texte, couleurs, bordures et arrière-plans ··· 17

13. Positionnement : relative et absolute ··· 18

14. Positionnement : fixed et sticky ··· 19

15. Mise en page avec float et clear ··· 20

16. Flexbox, affichage et débordement ··· 21

17. Liens, listes, tableaux et ombres ··· 22

18. Filtres et transformations 2D ··· 23

19. Transitions et animations ··· 24

JavaScript

20. Intégration, variables et types ··· 25

Sommaire — suite

JavaScript

21. Opérateurs, conversions et dialogues ··· 26
22. Conditions et choix multiples ··· 27
23. Boucles et accumulation ··· 28
24. TP : jeu de devinettes ··· 29
25. DOM : sélectionner et modifier des éléments ··· 30
26. DOM : lire et contrôler un formulaire ··· 31
27. Fonctions, portée et événements ··· 32
28. Nombres, chaînes et méthodes usuelles ··· 33
29. Tableaux et parcours ··· 34
30. Dates, temporisation et horloge numérique ··· 35
31. Compléments : Math, chaînes et sorties ··· 36
32. Médias, styles et dates de formulaire ··· 37

Bases de données et SQL

33. Notions relationnelles et contraintes ··· 38
34. SQL : commandes et types MySQL ··· 39
35. Création des tables : gestion des locations ··· 40
36. Modifier et supprimer une structure ··· 41
37. Insérer, modifier et supprimer des lignes ··· 42
38. Base commerciale : schéma des exercices ··· 43
39. SELECT, filtres et expressions ··· 44
40. Agrégats, tri et sous-requêtes ··· 45

Sommaire — suite

Bases de données et SQL

41. GROUP BY, HAVING et jointures ··· 46

42. Fonctions SQL sur les chaînes ··· 47

43. Fonctions SQL sur les dates et heures ··· 48

44. Jointures STI, mises à jour croisées et LCD ··· 49

PHP

45. Principe client-serveur et environnement ··· 50

46. Variables, types, constantes et opérateurs ··· 51

47. Formulaires : lecture, validation et affichage ··· 52

48. Conditions, boucles et fonctions ··· 53

49. Tableaux, parcours avec foreach et tris ··· 54

50. Chaînes et fonctions usuelles ··· 55

51. Dates et heures : timestamp et checkdate ··· 56

52. Dates et heures : date et getdate ··· 57

53. Dates et heures : time et mktime ··· 58

54. Dates et heures : strtotime et calculs ··· 59

55. MySQLi procédural : connexion et lecture ··· 60

56. MySQLi procédural : requêtes préparées ··· 61

57. TP : répertoire de sites en MySQLi procédural ··· 62

58. Fonctions complémentaires et superglobales ··· 63

59. MySQLi : types de résultats et fonctions fetch ··· 64

Travaux pratiques

60. Projet 1 : gestion des élèves — énoncé et base ··· 65

Sommaire — suite

Travaux pratiques

61. Projet 1 : formulaire et validation JavaScript ··· 66

62. Projet 1 : connexion et insertion PHP ··· 67

63. Projet 1 : liste, modification et statistiques ··· 68

64. Projet 2 : recharge électrique — énoncé et base ··· 69

65. Projet 2 : horaires et insertion paramétrée ··· 70

66. Projet 2 : statistiques, coûts et tests ··· 71

Annexes

67. Synthèse et contrôle des acquis ··· 72

68. Révision : questions et réponses essentielles ··· 73

69. Sources, périmètre et conventions de cette édition ··· 74

Structure d’un document HTML5

Ouvrir cette partie sur STI v2.0 → | QR de lecture en bas de page

Objectif

Créer un document HTML valide et comprendre le rôle de ses éléments principaux. HTML structure le contenu ; CSS gère sa présentation ; JavaScript ajoute des interactions.

Document de départ

```
<!DOCTYPE html>
<html lang="fr">
<head>
  <meta charset="utf-8">
  <meta name="viewport"
        content="width=device-width, initial-scale=1">
  <meta name="description" content="Cours de STI">
  <title>Ma première page</title>
  <link rel="stylesheet" href="style.css">
  <script src="script.js" defer></script>
</head>
<body>
  <h1>Bienvenue au Lycée Rafèha</h1>
  <p>Une page structurée en HTML5.</p>
</body>
</html>
```

Rôle des éléments

• <!DOCTYPE html> active le mode standard du navigateur.

• html est l’élément racine. lang="fr" indique la langue du contenu.

• head contient le titre et les métadonnées. title est affiché dans l’onglet du navigateur.

• meta charset="utf-8" définit l’encodage ; link relie la feuille de style.

• body contient le contenu de la page. Les couleurs et les images de fond se règlent en CSS, pas avec l’ancien attribut background.

• defer reporte l’exécution du script externe après l’analyse du HTML.

À réaliser

Enregistrer ce document sous index.html, créer style.css et script.js dans le même dossier, puis ouvrir la page dans le navigateur.

Syntaxe, attributs et imbrication

Ouvrir cette partie sur STI v2.0 → | QR de lecture en bas de page

Éléments et attributs

Une balise ouvrante peut porter des attributs. Les éléments non vides ont une balise fermante. Respecter l'imbrication : le dernier élément ouvert est le premier fermé.

```
<p class="important" id="intro" title="Présentation">
  Un texte <strong>important</strong>.
</p>
<a href="contact.html">Nous contacter</a>
<img src="lycee.jpg" alt="Entrée du lycée" width="400">
```

• id identifie un seul élément dans le document.

• class contient un ou plusieurs noms de classes séparés par des espaces.

• title apporte une information complémentaire ; il ne remplace pas un libellé accessible.

• style permet un style local ; privilégier une feuille CSS externe.

Éléments vides

br, hr, img, input, meta, link et source n'ont pas de balise fermante en HTML. Le slash final de
 est facultatif et ne transforme pas l'élément en paire de balises.

Casse et caractères réservés

Les noms des balises et des attributs HTML ne distinguent généralement pas les majuscules des minuscules. En revanche, les valeurs de id et de class doivent conserver leur casse pour les sélecteurs CSS et les accès JavaScript.

```
<p>Écrire &lt;p&gt; pour afficher le nom de la balise.</p>
<p>Utiliser &amp; pour afficher une esperluette.</p>
<!-- Ceci est un commentaire HTML -->
```

Bonnes pratiques

• Employer des noms simples : menu-principal, titre, carte.

• Indenter le code et entourer les valeurs d'attributs de guillemets droits.

• Ne pas imbriquer un paragraphe dans un autre paragraphe.

• Préférer les éléments sémantiques aux éléments génériques lorsque leur rôle correspond au contenu.

Application

Créer deux paragraphes de même classe et leur attribuer deux identifiants différents. Expliquer pourquoi deux éléments ne doivent pas partager le même id.

Texte, liens et contenus multimédias

Ouvrir cette partie sur STI v2.0 → | QR de lecture en bas de page

Structurer le texte

h1 à h6 représentent des titres hiérarchisés ; p un paragraphe ; br un saut de ligne ; hr une séparation thématique. Choisir un titre selon son niveau, et non selon sa taille visuelle.

• strong : importance forte ; em : emphase ; mark : passage mis en évidence.

• abbr : abréviation ; sup et sub : exposant et indice.

• blockquote : citation en bloc ; q : citation courte ; cite : titre d’une œuvre.

• del et ins : contenu supprimé ou ajouté ; pre et code : code préformaté.

• address : coordonnées de contact ; time : date ou heure.

```
<h1>Le Web</h1>
<h2>Les langages</h2>
<p><abbr title="HyperText Markup Language">HTML</abbr>
structure la page. CSS gère sa <em>présentation</em>.</p>
<p>H<sub>2</sub>O ; x<sup>2</sup></p>
<time datetime="2026-10-21">21 octobre 2026</time>
<a href="https://www.php.net/">Documentation PHP</a>
<a href="#suite">Aller à la suite</a>
<h2 id="suite">Suite du cours</h2>
```

Images, audio et vidéo

```
<figure>
  <img src="schema.png" alt="Échanges client-serveur">
  <figcaption>Le navigateur dialogue avec le serveur.</figcaption>
</figure>
<audio controls>
  <source src="cours.mp3" type="audio/mpeg">
  Votre navigateur ne peut pas lire ce fichier audio.
</audio>
<video controls width="480">
  <source src="demo.mp4" type="video/mp4">
  Votre navigateur ne peut pas lire cette vidéo.
</video>
```

À retenir

alt décrit une image informative ; alt="" convient à une image purement décorative. Pour une vidéo pédagogique, prévoir des sous-titres. Un lien doit avoir un intitulé compréhensible hors contexte.

Listes, tableaux et structure sémantique

Ouvrir cette partie sur STI v2.0 → | QR de lecture en bas de page

Les listes

```
<ul><li>HTML5</li><li>CSS3</li></ul>
<ol><li>Créer la page</li><li>Tester</li></ol>
<dl>
  <dt>DOM</dt><dd>Représentation de la page en objets.</dd>
</dl>
```

ul crée une liste non ordonnée ; ol une liste ordonnée ; li un élément de liste. dl regroupe des termes dt et leurs descriptions dd.

Les tableaux de données

```
<table>
  <caption>Résultats des élèves</caption>
  <thead>
    <tr><th scope="col">Nom</th><th scope="col">Note</th></tr>
  </thead>
  <tbody>
    <tr><td>Amal</td><td>16</td></tr>
    <tr><td>Yassine</td><td>14</td></tr>
  </tbody>
</table>
```

tr définit une ligne ; th une cellule d’en-tête ; td une cellule de données. thead, tbody et tfoot regroupent les lignes. colspan et rowspan permettent des fusions de cellules. Ne pas utiliser un tableau pour mettre en page tout le site.

Organiser une page

```
<header><h1>Mon site</h1></header>
<nav><a href="index.html">Accueil</a></nav>
<main>
  <section>
    <h2>Actualités</h2>
    <article><h3>Projet STI</h3><p>Présentation...</p></article>
  </section>
  <aside>Informations complémentaires</aside>
</main>
<footer>Contact du lycée</footer>
```

header : en-tête ; nav : navigation ; main : contenu principal ; section : section thématique ; article : contenu autonome ; aside : complément ; footer : pied de page.

Éléments génériques

Par défaut, div est affiché en bloc et span en ligne. Ils servent à regrouper du contenu sans lui attribuer de signification sémantique particulière. Ils restent utiles même sans attribut CSS.

Formulaires : saisie et transmission

Ouvrir cette partie sur STI v2.0 → | QR de lecture en bas de page

Formulaire complet

```
<form id="inscription" method="post" action="inscrire.php">
  <fieldset>
    <legend>Identification</legend>
    <label for="nom">Nom</label>
    <input type="text" id="nom" name="nom"
           maxlength="50" required>
    <label for="email">Adresse électronique</label>
    <input type="email" id="email" name="email" required>
    <label for="naissance">Date de naissance</label>
    <input type="date" id="naissance" name="naissance">
    <label for="classe">Classe</label>
    <select id="classe" name="classe">
      <option value="4SI1">4ème SI 1</option>
      <option value="4SI2">4ème SI 2</option>
    </select>
    <label for="message">Message</label>
    <textarea id="message" name="message" rows="4"></textarea>
    <button type="submit">Envoyer</button>
    <button type="reset">Réinitialiser</button>
  </fieldset>
</form>
```

Comprendre les attributs

• action désigne l’URL de traitement ; method choisit get ou post. Leur présence explicite améliore la lisibilité, sans être toujours obligatoire en HTML.

• name est la clé utilisée pour transmettre la valeur au serveur ; id sert notamment à l’association avec label for.

• required rend la saisie obligatoire côté navigateur ; maxlength limite la longueur ; placeholder donne une indication, sans remplacer label.

• min, max et step règlent les bornes et le pas d’un champ number.

• readonly interdit la modification mais conserve l’envoi ; disabled désactive le champ et l’exclut de l’envoi.

GET ou POST ?

GET place les paramètres dans l’URL et convient notamment aux recherches. POST transmet les données dans le corps de la requête et convient aux modifications. POST ne chiffre pas les données : utiliser HTTPS.

Important

Les validations HTML et JavaScript facilitent la saisie mais ne remplacent jamais la validation sur le serveur PHP.

Formulaires : options et aide-mémoire

Ouvrir cette partie sur STI v2.0 → | QR de lecture en bas de page

Cases à cocher et boutons radio

```
<fieldset>
  <legend>Langages étudiés</legend>
  <label><input type="checkbox" name="langages[]"
                value="html" checked> HTML</label>
  <label><input type="checkbox" name="langages[]"
                value="php"> PHP</label>
</fieldset>
<fieldset>
  <legend>Groupe</legend>
  <label><input type="radio" name="groupe"
                value="A" required> A</label>
  <label><input type="radio" name="groupe" value="B"> B</label>
</fieldset>
```

Les radios de même name sont exclusifs. Les cases à cocher autorisent plusieurs choix ; elles peuvent partager un nom de tableau, par exemple langages[], pour leur traitement en PHP. Une case non cochée n'est normalement pas transmise.

Liste multiple et groupes

```
<select name="options[]" multiple size="4">
  <optgroup label="Web">
    <option value="html" selected>HTML</option>
    <option value="css">CSS</option>
  </optgroup>
</select>
<input type="tel" name="telephone" pattern="[0-9]{8}">
<input type="number" name="note" min="0" max="20" step="0.5">
```

Autres types de champs

• password : saisie masquée, mais non chiffrée par ce seul attribut.

• search : recherche ; url : adresse Web ; tel : téléphone.

• range : curseur ; color : couleur ; date : date ; file : fichier.

• hidden : valeur invisible dans l'interface, mais modifiable par l'utilisateur.

Envoi de fichier

Utiliser method="post" et enctype="multipart/form-data" sur form, ainsi qu'un input type="file". PHP récupère le fichier via $_FILES. Vérifier côté serveur sa taille, son type réel et son nom de destination.

Application

Construire un formulaire avec une adresse électronique, une note de 0 à 20, un groupe et plusieurs langages. Vérifier les données transmises dans les outils réseau du navigateur.

Cadres intégrés et cibles de navigation

Ouvrir cette partie sur STI v2.0 → | QR de lecture en bas de page

iframe : intégrer un document

iframe affiche une page dans une autre. title fournit un nom accessible au cadre ; name permet de le désigner comme cible. id et name n’ont pas le même rôle.

```
<nav>
  <a href="inscription.html" target="cadre">Inscription</a>
  <a href="liste.php" target="cadre">Liste des élèves</a>
</nav>
<iframe name="cadre" id="cadre" title="Espace de travail"
        src="inscription.html"></iframe>

iframe { width: 100%; min-height: 500px; border: 0; }
```

target sur les liens et formulaires

• _self : contexte courant ; valeur usuelle par défaut.

• _blank : nouveau contexte, généralement un nouvel onglet.

• _parent : contexte parent ; _top : contexte de navigation supérieur.

• cadre : contexte portant ce nom, ici l’iframe de la page.

```
<form action="ajouter.php" method="post" target="cadre">
  <label for="nom">Nom</label>
  <input id="nom" name="nom" required>
  <button type="submit">Ajouter</button>
</form>
<a href="https://www.php.net/" target="_blank"
   rel="noopener">Documentation PHP</a>
```

Si le formulaire est déjà affiché dans cadre, _self permet aussi d’afficher sa réponse dans ce cadre. Si aucun contexte nommé ne correspond à une cible, le navigateur peut en ouvrir un nouveau.

Sécurité et limites

Une page tierce peut interdire son intégration par des en-têtes de sécurité. La politique de même origine limite les accès JavaScript entre documents de domaines différents. sandbox permet de restreindre un cadre ; ses permissions doivent être choisies selon le besoin. Ne pas ajouter toutes les permissions sans les comprendre.

Application

Créer index.html avec une navigation et un cadre nommé cadre. Afficher successivement deux pages locales. Tester ensuite un formulaire soumis vers ajouter.php sur un serveur PHP.

Attributs globaux, datalist et événements HTML

Ouvrir cette partie sur STI v2.0 → | QR de lecture en bas de page

Attributs globaux utiles

id identifie ; class regroupe ; title apporte une information ; lang indique une langue ; hidden masque un contenu ; data-* stocke des données propres à l'application. tabindex influence la navigation au clavier : éviter les valeurs positives pour ne pas perturber l'ordre naturel.

```
<article class="produit" data-code="P01" lang="fr">
  <h2>Un produit</h2>
</article>
<label for="ville">Ville</label>
<input id="ville" name="ville" list="villes">
<datalist id="villes">
  <option value="Tunis"></option>
  <option value="Sfax"></option>
  <option value="Sousse"></option>
</datalist>
```

datalist propose des suggestions sans interdire une autre saisie. select propose une liste de choix. Pour imposer une ville reconnue, contrôler sa valeur côté serveur.

Événements dans les attributs

```
<button type="button" onclick="saluer()">Bonjour</button>
<form action="traitement.php" method="post"
      onsubmit="return valider()">
  <input name="nom" required>
  <button type="submit">Envoyer</button>
</form>
```

Dans cet exemple, le return de onsubmit transmet le résultat de valider : false annule l'action par défaut. Écrire seulement onsubmit="valider()" ne transmet pas automatiquement ce false. Avec addEventListener, employer event.preventDefault() pour bloquer l'envoi.

Correspondance des noms

onclick correspond à click ; onchange à change ; oninput à input ; onsubmit à submit. input réagit au changement pendant la saisie ; change intervient lors de la validation d'un changement selon le contrôle. Éviter de multiplier les gestionnaires dans le HTML : addEventListener sépare mieux structure et comportement.

Application

Construire une zone de saisie avec suggestions et un message de validation. Conserver un libellé visible ; ne pas faire dépendre les consignes uniquement de title ou de placeholder.

Syntaxe et intégration des styles

Ouvrir cette partie sur STI v2.0 → | QR de lecture en bas de page

Définition

CSS (Cascading Style Sheets) décrit la présentation des éléments HTML. Une règle associe un sélecteur à des déclarations propriété: valeur.

```
p {
  color: #16324f;
  font-size: 1rem;
  line-height: 1.6;
}
```

Trois modes d’intégration

```
<!-- Feuille externe : méthode à privilégier -->
<link rel="stylesheet" href="style.css">
<!-- Style interne, dans head -->
<style>
  h1 { color: navy; }
</style>
<!-- Style local -->
<p style="color: navy;">Un paragraphe</p>
```

Cascade, héritage et priorité

Plusieurs règles peuvent viser un même élément. Pour les déclarations ordinaires d’une même origine et d’une même couche, la spécificité du sélecteur puis l’ordre d’écriture départagent les règles concurrentes. Un identifiant est plus spécifique qu’une classe, qui est plus spécifique qu’un nom de balise.

Certaines propriétés sont héritées, comme color et font-family. Les marges et les bordures ne le sont généralement pas. Éviter l’emploi systématique de !important.

```
body { font-family: Arial, sans-serif; color: #222; }
p { color: blue; }
.important { color: darkred; }
#intro { color: darkgreen; }
/* Un p ayant class="important" et id="intro" sera vert. */
```

Règles de syntaxe

• Terminer chaque déclaration par un point-virgule.

• Ne pas séparer la valeur et son unité : 150px, et non 150 px.

• Écrire les commentaires entre /* et */.

• Une règle CSS ordinaire commence par un sélecteur ; les règles commençant par @, comme @media, suivent une syntaxe particulière.

Application

Relier une feuille externe à la page d’accueil et définir les styles du corps, des titres et des paragraphes.

Sélecteurs, classes et identifiants

Ouvrir cette partie sur STI v2.0 → | QR de lecture en bas de page

Principaux sélecteurs

• p : tous les paragraphes ; * : tous les éléments.

• .important : les éléments portant la classe important.

• #intro : l'élément portant l'identifiant intro.

• h1, h2 : réunion de plusieurs sélecteurs.

• nav a : liens descendants de nav ; nav > a : liens enfants directs.

• input[type="email"] : champs ayant cet attribut et cette valeur.

• a:hover : lien survolé ; input:focus : champ ayant le focus.

```
<h1 id="titre" class="grand souligne">Le Web</h1>
<p class="important">À retenir</p>
<p class="grand important">Un autre paragraphe</p>

#titre { color: #16324f; }
.grand { font-size: 1.5rem; }
.souligne { text-decoration: underline; }
.important { font-weight: bold; }
p.important { border-left: 3px solid teal; }
a:hover { color: darkred; }
input:focus { outline: 2px solid #167d8d; }
```

Bien choisir id et class

Un id doit être unique dans le document. Une classe est réutilisable et un élément peut porter plusieurs classes. Pour des noms faciles à manipuler dans ce cours, utiliser des lettres minuscules, des chiffres après la première lettre et des tirets.

Les valeurs de class et de id sont sensibles à la casse : .important ne désigne pas class="Important". Les règles complètes des identifiants CSS autorisent davantage de caractères que cette convention simplifiée.

Sélecteurs combinés

```
.carte .titre { color: navy; } /* descendant */
.carte.titre { color: teal; } /* les deux classes */
li:first-child { font-weight: bold; }
tr:nth-child(even) { background-color: #eef4f7; }
```

Application

Créer trois cartes partageant la classe carte. Ajouter une classe selectionnee à une seule carte et lui donner une bordure différente.

Unités et modèle de boîte

Ouvrir cette partie sur STI v2.0 → | QR de lecture en bas de page

Unités absolues et relatives

• px : pixel CSS ; pt : point ; cm, mm et in : unités physiques.

• rem : taille de police de l’élément racine.

• em : taille de police de l’élément ; pour font-size, référence à la taille héritée.

• % : référence dépendant de la propriété, souvent une dimension du conteneur.

• vw et vh : centième de la largeur ou de la hauteur de la zone d’affichage.

• ch : mesure liée à la largeur du caractère zéro ; ex : hauteur d’x de la police.

16px est une taille courante par défaut, pas une égalité universelle pour 1em.

Les quatre zones d’une boîte

Du centre vers l’extérieur : contenu, padding, border, margin.

Avec box-sizing: content-box, width désigne la largeur du contenu. La largeur extérieure occupée, hors cas particuliers de mise en page, vaut :

Largeur = width + padding gauche/droit + border gauche/droit + margin gauche/droit.

```
.boite {
  width: 200px;
  padding: 10px;
  border: 2px solid teal;
  margin: 15px;
}
/* Boîte bordure comprise : 224px ; avec marges : 254px. */
```

Le modèle border-box

```
* { box-sizing: border-box; }
.boite {
  width: 200px;
  padding: 10px;
  border: 2px solid teal;
  margin: 15px;
}
/* Boîte bordure comprise : 200px ; avec marges : 230px. */
```

Notation courte

margin: 1em 0 2em 0.5em suit l’ordre haut, droite, bas, gauche. Deux valeurs définissent les côtés verticaux puis horizontaux. Une valeur s’applique aux quatre côtés. Les mêmes règles valent pour padding.

Attention

Certaines marges verticales de blocs peuvent fusionner. Pour une boîte en ligne non remplacée comme span, width et height ne s’appliquent pas comme sur un bloc ; utiliser inline-block si nécessaire.

Texte, couleurs, bordures et arrière-plans

Ouvrir cette partie sur STI v2.0 → | QR de lecture en bas de page

Texte et affichage

```
body {
  font-family: Arial, sans-serif;
  font-size: 1rem;
  line-height: 1.6;
  color: #16324f;
  background-color: #f4f7fa;
}
h1 { font-size: 2rem; text-align: center; }
.note { font-weight: bold; font-style: italic; }
.cache { display: none; }
.etiquette { display: inline-block; padding: 0.3rem; }
```

color règle la couleur du texte ; background-color celle du fond. Une couleur peut être un nom, une valeur hexadécimale, rgb() ou hsl(). opacity affecte l'élément entier et ses descendants ; une couleur avec alpha permet de ne rendre transparent que le fond.

Bordures et fonds

```
.carte {
  border: 2px solid #167d8d;
  border-radius: 8px;
  padding: 1rem;
  margin: 1rem 0;
  background: white;
}
.banniere {
  background: #16324f url("fond.png") no-repeat center;
  background-size: cover;
}
.degrade { background: linear-gradient(90deg, teal, navy); }
```

border-top, border-right, border-bottom et border-left ciblent un côté. border-width, border-style et border-color règlent séparément épaisseur, style et couleur. Exemples de styles : solid, dashed, dotted, double, inset.

background-repeat prend notamment repeat, no-repeat, repeat-x ou repeat-y ; background-attachment: fixed fixe l'image au viewport dans les situations usuelles ; background-position règle sa position. radial-gradient() crée un dégradé radial.

Application : modèle de boîte

Créer un div de largeur 400px, avec un padding supérieur de 40px, une bordure verte de 10px et une marge de 30px. Comparer sa largeur extérieure avec et sans border-box.

Vérification

Utiliser l'onglet « Éléments » du navigateur pour inspecter les styles calculés et le schéma du modèle de boîte.

Positionnement : relative et absolute

Ouvrir cette partie sur STI v2.0 → | QR de lecture en bas de page

Flux normal et propriété position

Dans le flux normal, les éléments occupent leur place selon leur type d'affichage. position: static est la valeur initiale. top, right, bottom et left servent à décaler des éléments positionnés.

Position relative

```
.etiquette {
  position: relative;
  top: 5px;
  left: 10px;
}
```

L'élément est décalé visuellement, mais sa place initiale reste réservée dans le flux. Il peut recouvrir un voisin sans que celui-ci se déplace.

Position absolue

```
<div class="carte">
  <img src="fleur.jpg" alt="Bouquet de fleurs">
  <span class="badge">Nouveau</span>
</div>

.carte { position: relative; width: 300px; }
.carte img { display: block; width: 100%; }
.badge {
  position: absolute;
  top: 10px;
  right: 10px;
  padding: 0.4rem;
  color: white;
  background: #147c89;
}
```

Un élément absolute est retiré du flux. Dans le cas courant illustré ici, il se place par rapport au plus proche ancêtre positionné, la carte. Sans cet ancêtre, sa référence peut être le bloc conteneur initial. Certaines propriétés comme transform peuvent aussi créer un bloc de référence.

Empilement

z-index permet de régler l'ordre d'empilement dans les contextes où il s'applique. Une valeur très élevée ne permet pas toujours de dépasser un élément appartenant à un autre contexte d'empilement.

Application

Afficher un prix dans le coin inférieur droit d'une image. Retirer position: relative de la carte et expliquer le changement. Éviter de positionner tous les blocs en absolute pour construire une page entière.

Positionnement : fixed et sticky

Ouvrir cette partie sur STI v2.0 → | QR de lecture en bas de page

fixed : position généralement liée à la fenêtre

```
.aide {
  position: fixed;
  bottom: 1rem;
  right: 1rem;
  padding: 0.7rem;
  background: #16324f;
  color: white;
  z-index: 10;
}
```

L’élément est retiré du flux et reste généralement visible au défilement, relativement au viewport. Un ancêtre transformé peut modifier cette référence. Prévoir un espace suffisant pour ne pas cacher du contenu, notamment sur mobile.

sticky : ancrage pendant le défilement

```
.entete-table {
  position: sticky;
  top: 0;
  background: white;
  z-index: 2;
}
```

L’élément reste dans le flux. Il se comporte comme un élément normal jusqu’à atteindre le seuil fixé par top, puis reste ancré dans les limites de son contexte de défilement et de son conteneur. Sans seuil pertinent, l’effet attendu peut ne pas apparaître.

Comparaison

• static : flux normal, sans décalage par les propriétés de position.

• relative : place conservée, décalage visuel possible.

• absolute : hors flux, référence à un bloc conteneur.

• fixed : hors flux, référence généralement au viewport.

• sticky : place conservée, ancrage conditionné par le défilement.

Points de contrôle

Un ancêtre ayant overflow: auto ou hidden peut modifier le comportement de sticky. Une zone trop courte ne laisse pas assez d’espace pour observer l’ancrage. Vérifier aussi que le contenu reste accessible au clavier et à l’agrandissement du texte.

Application

Créer un en-tête sticky et un bouton d’aide fixed. Tester le défilement, le zoom et une fenêtre étroite. Vérifier qu’aucun champ de formulaire n’est caché par l’interface fixe.

Mise en page avec float et clear

Ouvrir cette partie sur STI v2.0 → | QR de lecture en bas de page

Flottants

float: left place un élément à gauche et permet au contenu suivant de l’entourer. float: right le place à droite. clear évite qu’un élément reste à côté de flottants antérieurs.

```
<header><img class="logo" src="logo.png" alt="Logo du lycée">
  <h1>Gestion des élèves</h1></header>
<div class="milieu">
  <nav><a href="inscription.html" target="cadre">Inscription</a></nav>
  <section class="contenu">
    <iframe name="cadre" title="Travail"
            src="inscription.html"></iframe>
  </section>
</div>
<footer>Lycée Rafèha</footer>

* { box-sizing: border-box; }
.logo { float: left; width: 70px; margin-right: 1rem; }
header { display: flow-root; }
nav { float: left; width: 180px; }
nav a { display: block; padding: 0.6rem; }
.contenu { margin-left: 200px; }
.milieu { display: flow-root; }
iframe { width: 100%; min-height: 500px; border: 0; }
footer { clear: both; text-align: center; }
```

Contenir les flottants

Un conteneur dont tous les enfants flottent peut ne pas les inclure dans sa hauteur ordinaire. display: flow-root crée un contexte de formatage qui contient les flottants. clear: both peut faire commencer le pied de page sous les flottants précédents.

Contraintes de l’activité

Les projets « Gestion des élèves » et « Recharge électrique » de la plateforme travaillent explicitement float et clear. Dans ces exercices, respecter cette consigne plutôt que remplacer immédiatement la mise en page par flex.

Application

Construire une navigation de largeur fixe et une zone de contenu à droite. Ajouter une transition aux liens. Tester les débordements lorsque la fenêtre devient étroite ; proposer ensuite une règle adaptée aux petits écrans.

Flexbox, affichage et débordement

Ouvrir cette partie sur STI v2.0 → | QR de lecture en bas de page

Affichage

block occupe généralement une ligne disponible ; inline s’intègre au texte ; inline-block combine un placement en ligne et une boîte dimensionnable. display: none retire l’élément de l’affichage et de la mise en page ; visibility: hidden le masque en conservant sa place.

Conteneur flexible

```
<div class="cartes">
  <article>HTML5</article>
  <article>CSS3</article>
  <article>JavaScript</article>
</div>
```

```
.cartes {
  display: flex;
  flex-direction: row;
  flex-wrap: wrap;
  gap: 1rem;
  justify-content: space-between;
  align-items: stretch;
}
.cartes article {
  flex: 1 1 180px;
  padding: 1rem;
  border: 1px solid teal;
}
```

justify-content agit sur l’axe principal ; align-items sur l’axe transversal. Changer flex-direction change ces axes. gap ajoute un espace entre les éléments. flex-wrap autorise le retour à la ligne.

Débordement

```
.zone {
  max-height: 200px;
  overflow: auto;
}
.texte-long { overflow-wrap: anywhere; }
```

overflow: visible laisse déborder ; hidden masque le débordement ; auto ajoute un mécanisme de défilement si nécessaire ; scroll le prévoit même si le contenu ne déborde pas dans les situations usuelles. Vérifier que masquer un débordement ne rend pas une information inaccessible.

Application

Afficher trois cartes flexibles qui passent sur plusieurs lignes. Construire ensuite le même écran avec float pour comparer les mécanismes. L’emploi de Flexbox est un complément ; il ne remplace pas la consigne float d’un TP qui l’impose.

Liens, listes, tableaux et ombres

Ouvrir cette partie sur STI v2.0 → | QR de lecture en bas de page

États des liens et focus

```
a:link { color: navy; }
a:visited { color: purple; }
a:hover { color: darkred; }
a:active { color: orange; }
a:focus-visible { outline: 3px solid teal; }
```

L'ordre link, visited, hover, active aide à éviter des conflits courants à spécificité égale. Le navigateur restreint les styles de visited pour protéger la confidentialité. Ne pas supprimer l'indicateur de focus sans solution de remplacement.

Listes

```
ul { list-style-type: square; list-style-position: outside; }
.menu { list-style: none; padding-left: 0; }
.illustrations { list-style-image: url("puce.png"); }
```

list-style combine type, position et image. Lorsqu'une image de puce valide est disponible, elle est utilisée à la place du type de puce.

Tableaux

```
table { width: 100%; border-collapse: collapse; }
th, td { border: 1px solid #9aabb8; padding: 0.5rem; }
th { background: #e8f3f5; text-align: left; }
tr:nth-child(even) { background: #f4f7fa; }
```

border-collapse: collapse fusionne les bordures voisines. table-layout: fixed fixe davantage le calcul des largeurs à partir de la table et de ses colonnes ; gérer les contenus longs.

Ombres et arrondis

```
.carte {
  border-radius: 12px 0 12px 0;
  box-shadow: 0 4px 12px rgb(0 0 0 / 20%);
}
h1 { text-shadow: 1px 2px 3px #b7c4cc; }
```

box-shadow : décalage horizontal, vertical, flou, puis éventuellement étalement et couleur. text-shadow s'applique au texte. Des valeurs multiples séparées par des virgules produisent plusieurs ombres. Conserver un contraste et une lisibilité suffisants.

Filtres et transformations 2D

Ouvrir cette partie sur STI v2.0 → | QR de lecture en bas de page

Filtres étudiés

```
.flou { filter: blur(3px); }
.gris { filter: grayscale(100%); }
.inverse { filter: invert(100%); }
.effet { filter: grayscale(100%) blur(1px); }
```

blur utilise une longueur positive ou nulle ; grayscale et invert admettent notamment un pourcentage de 0 % à 100 %. Leur valeur neutre est 0 ou 0 %. L’ordre des fonctions peut modifier le résultat. Le filtre s’applique au rendu de l’élément, y compris son contenu ; il ne modifie pas le fichier source.

Transformations

```
.rotation { transform: rotate(15deg); }
.taille { transform: scale(1.1); }
.decalage { transform: translate(20px, 10px); }
.inclinaison { transform: skew(10deg, 0deg); }
.combine {
  transform-origin: center;
  transform: translateX(15px) rotate(10deg) scale(1.05);
}
```

rotate tourne ; scale change l’échelle ; translate déplace ; skew incline. transform-origin fixe le point d’origine des transformations. Les fonctions se composent : modifier leur ordre peut changer l’effet observé.

Transform et mise en page

Une transformation visuelle ne redistribue pas le flux comme une modification des dimensions ordinaires. L’élément peut donc chevaucher un voisin. Un élément transformé peut aussi créer un nouveau contexte d’empilement et un bloc de référence pour certains descendants positionnés.

Survol progressif

```
.photo { transition: transform 0.3s, filter 0.3s; }
.photo:hover {
  transform: scale(1.05);
  filter: grayscale(100%);
}
```

Application

Créer une carte de fleur qui grandit légèrement au survol, puis tester blur, grayscale et invert séparément. Ajouter la même mise en évidence au focus si la carte est interactive ; ne pas réserver une information indispensable au seul survol.

Transitions et animations

Ouvrir cette partie sur STI v2.0 → | QR de lecture en bas de page

Transition entre deux états

Syntaxe simplifiée : propriété durée fonction-de-temporisation délai.

```
nav a {
  display: block;
  background-color: #edf5f6;
  transition: background-color 0.3s ease,
              transform 0.3s ease-out;
}
nav a:hover, nav a:focus-visible {
  background-color: #bfe5e9;
  transform: translateX(5px);
}
```

linear garde une progression régulière ; ease accélère puis ralentit ; ease-in privilégie un départ lent ; ease-out une fin lente ; ease-in-out combine les deux. Éviter transition: all si seules quelques propriétés doivent changer.

Animation avec étapes

```
@keyframes apparition {
  from { opacity: 0; transform: translateY(15px); }
  to { opacity: 1; transform: translateY(0); }
}
.carte {
  animation: apparition 0.6s ease-out 0s 1 normal both;
}
```

Les valeurs abrégées indiquent ici nom, durée, temporisation, délai, nombre d’itérations, sens et mode de remplissage. Des étapes 0 %, 50 % et 100 % permettent plusieurs états. animation-iteration-count: infinite répète sans fin ; éviter les mouvements permanents inutiles.

Réduire les mouvements

```
@media (prefers-reduced-motion: reduce) {
  .carte { animation: none; }
  nav a { transition: none; }
}
```

Application

Faire apparaître une carte une seule fois. Ajouter une transition de bordure aux champs ayant le focus. Vérifier le comportement lorsque l’utilisateur préfère moins d’animations. Les effets visuels ne doivent pas retarder la lecture ou masquer les erreurs de formulaire.

Intégration, variables et types

Ouvrir cette partie sur STI v2.0 → | QR de lecture en bas de page

Rôle et chargement

Dans ce cours, JavaScript s’exécute dans le navigateur : il réagit aux événements, manipule le DOM et peut échanger avec un serveur. Il existe aussi des environnements JavaScript côté serveur.

```
<script src="script.js" defer></script>
<!-- Ou script interne placé à la fin de body -->
<script>
  console.log("Page prête");
</script>
```

Déclarer avec let

```
let nombre = 2;
let texte = "Bonjour";
let actif = true;
let resultat;                 // undefined
let selection = null;
nombre = nombre + 3;
console.log(nombre);          // 5
```

let déclare une variable dont la portée est le bloc délimité par des accolades. Il faut la déclarer avant de l’utiliser et ne pas la redéclarer dans le même bloc. Le cours utilise let pour les variables ; const reste possible pour une liaison qui ne sera pas réaffectée.

```
let message = "À l'extérieur";
if (actif) {
  let message = "Dans le bloc";
  console.log(message);
}
console.log(message);         // À l'extérieur
```

Types et identifiants

Les types usuels comprennent number, string, boolean, undefined, object et bigint ; null représente explicitement une absence de valeur. Les tableaux sont des objets. typeof null vaut "object", particularité historique du langage.

Un identifiant est sensible à la casse, ne commence pas par un chiffre et ne peut pas être un mot réservé. Préférer des noms explicites : note, total, nomEleve.

Commentaires et instructions

// commence un commentaire sur une ligne ; /* ... */ un commentaire de plusieurs lignes. Terminer les instructions par un point-virgule dans les exemples du cours.

Application

Déclarer le nom d’un élève, sa moyenne et son état d’admission, puis afficher leurs types avec typeof.

Opérateurs, conversions et dialogues

Ouvrir cette partie sur STI v2.0 → | QR de lecture en bas de page

Calcul et concaténation

```
let a = 7, b = 2;
console.log(a + b, a - b, a * b); // 9, 5, 14
console.log(a / b, a % b, a ** b); // 3.5, 1, 49
let prenom = "Amal";
let message = "Bonjour " + prenom;
console.log(message);
console.log(4 + "2");            // "42"
console.log(Number("4") + 2);     // 6
```

+=, -=, *= et /= combinent une opération et une affectation. ++ ajoute une unité ; -- soustrait une unité.

Dialogues du navigateur

alert(message) affiche un message. prompt(message) renvoie une chaîne ou null en cas d'annulation. confirm(message) renvoie true pour OK et false pour Annuler.

```
let saisie = prompt("Saisir une note de 0 à 20 :");
if (saisie !== null && saisie.trim() !== "") {
  let note = Number(saisie);
  if (Number.isFinite(note) && note >= 0 && note <= 20) {
    alert("Note : " + note);
  } else {
    alert("Note invalide.");
  }
}
```

Convertir correctement

• Number("12.5") produit 12.5 ; Number("12abc") produit NaN.

• Number("") produit 0 : vérifier une saisie vide avant conversion.

• Number.parseInt("12", 10) analyse un entier en base 10.

• Number.parseFloat("12.5abc") produit 12.5 : cette fonction ne valide pas toute la chaîne.

• String(42) produit "42" ; Number.isNaN(x) teste la valeur NaN sans conversion.

La base 10 explicite de parseInt rend l'intention claire. Sans base, l'analyse moderne utilise généralement 10, sauf notamment un préfixe 0x interprété en base 16.

Application

Demander deux nombres, gérer l'annulation et les chaînes vides, puis afficher leur somme après validation.

Conditions et choix multiples

Ouvrir cette partie sur STI v2.0 → | QR de lecture en bas de page

Comparaisons et logique

=== compare la valeur et le type ; !== teste leur différence. == et != peuvent convertir les opérandes : privilégier les comparaisons strictes. Les autres comparaisons sont <, <=, > et >=.

&& exprime ET ; || exprime OU ; ! exprime NON. Ces opérateurs utilisent des évaluations booléennes, mais && et || peuvent renvoyer un opérande non booléen.

```
let note = 14;
let resultat;
if (note < 0 || note > 20) {
  resultat = "Note invalide";
} else if (note >= 10) {
  resultat = "Admis";
} else {
  resultat = "Non admis";
}
console.log(resultat);
```

La variable resultat est déclarée avant les blocs, car elle est utilisée ensuite. Une variable déclarée avec let à l'intérieur d'un seul bloc ne serait pas accessible à cet endroit.

switch

```
let fruit = "Pommes";
switch (fruit) {
  case "Oranges":
    console.log("2 DT le kilo");
    break;
  case "Pommes":
    console.log("3,5 DT le kilo");
    break;
  case "Bananes":
  case "Cerises":
    console.log("9,5 DT le kilo");
    break;
  default:
    console.log("Fruit non disponible");
}
```

switch compare strictement l'expression aux valeurs des case. break évite de poursuivre dans le cas suivant.

Opérateur conditionnel

let etat = note >= 10 ? "Admis" : "Non admis";

Cette expression choisit l'une des deux valeurs ; la réserver à des choix simples pour garder un code lisible.

Boucles et accumulation

Ouvrir cette partie sur STI v2.0 → | QR de lecture en bas de page

for : nombre d’itérations connu

```
let somme = 0;
for (let i = 1; i <= 10; i++) {
  somme += i;
}
console.log(somme); // 55 ; i n'est pas accessible ici.
```

La condition du for est une condition de poursuite : le bloc s’exécute tant qu’elle est vraie.

while : test avant l’exécution

```
let nombre = 1;
while (nombre < 10) {
  nombre++;
}
console.log(nombre); // 10
```

do...while : au moins une exécution

```
let total = 0;
let continuer = true;
do {
  let saisie = prompt("Nombre positif ; négatif pour finir :");
  if (saisie === null) { break; }
  let nombre = Number(saisie);
  if (saisie.trim() === "" || !Number.isFinite(nombre)) {
    alert("Saisie invalide");
    continue;
  }
  continuer = nombre >= 0;
  if (continuer) { total += nombre; }
} while (continuer);
alert("Somme : " + total);
```

continue passe à l’itération suivante ; break quitte la boucle. continuer est initialisé à true : une saisie invalide fait recommencer la boucle sans modifier le total. Le nombre négatif de fin n’est pas ajouté à la somme.

Compter des prénoms

```
let prenoms = [], prenom;
while ((prenom = prompt("Prénom ; vide pour terminer :")) !== null) {
  if (prenom.trim() === "") { break; }
  prenoms.push(prenom.trim());
}
alert(prenoms.length + " prénoms : " + prenoms.join(", "));
```

Vigilance

Prévoir une progression et une condition d’arrêt. Une boucle synchrone infinie bloque l’interface du navigateur.

TP : jeu de devinettes

Ouvrir cette partie sur STI v2.0 → | QR de lecture en bas de page

Objectif

Générer un entier entre 1 et 12 puis permettre à l'utilisateur de le deviner. Compter seulement les propositions valides et autoriser l'annulation.

```
let minimum = 1;
let maximum = 12;
let secret = Math.floor(Math.random() *
             (maximum - minimum + 1)) + minimum;
let essais = 0;
let trouve = false;
let indice = "";
while (!trouve) {
  let saisie = prompt("Entier de 1 à 12. " + indice);
  if (saisie === null) {
    alert("Partie annulée.");
    break;
  }
  let nombre = Number(saisie);
  if (saisie.trim() === "" || !Number.isInteger(nombre)
      || nombre < minimum || nombre > maximum) {
    indice = "La proposition doit être un entier valide.";
    continue;
  }
  essais++;
  if (nombre > secret) {
    indice = "Cherchez plus petit.";
  } else if (nombre < secret) {
    indice = "Cherchez plus grand.";
  } else {
    trouve = true;
    alert("Bravo ! Trouvé en " + essais + " essai(s).");
  }
}
```

Comprendre le tirage

Math.random() renvoie un réel dans [0, 1[. Math.floor(x) renvoie le plus grand entier inférieur ou égal à x. Le calcul multiplie par le nombre de valeurs possibles, puis ajoute la borne minimale.

Questions

1. Pourquoi ne faut-il pas utiliser Math.round() dans cette formule ?

2. À quel endroit la portée de la variable nombre se termine-t-elle ?

3. Que devient essais lorsque l'utilisateur saisit une lettre ?

Améliorations

Limiter le nombre d'essais, proposer une nouvelle partie et remplacer prompt par un formulaire HTML. Math.random() ne convient pas à la génération de secrets de sécurité.

DOM : sélectionner et modifier des éléments

Ouvrir cette partie sur STI v2.0 → | QR de lecture en bas de page

Le DOM

Le Document Object Model représente le document sous forme de nœuds et d'objets. JavaScript peut sélectionner des éléments, lire leurs propriétés et modifier leur contenu.

• document.getElementById("intro") : élément d'identifiant intro, ou null.

• document.querySelector(".carte") : premier élément correspondant, ou null.

• document.querySelectorAll(".carte") : NodeList de tous les éléments correspondants.

• getElementsByTagName("p") et getElementsByClassName("carte") : collections dynamiques.

• getElementsByName("groupe") : collection des éléments portant ce name.

```
<p id="intro">Un texte initial.</p>
<img id="photo" src="avant.png" alt="Version initiale">

let paragraphe = document.getElementById("intro");
if (paragraphe !== null) {
  console.log(paragraphe.textContent);
  paragraphe.textContent = "Un autre texte";
  paragraphe.classList.add("important");
}
let photo = document.querySelector("#photo");
photo.setAttribute("src", "apres.png");
photo.alt = "Nouvelle version";
```

textContent ou innerHTML ?

textContent traite la valeur comme du texte. innerHTML lit ou interprète du balisage HTML. Ne pas affecter à innerHTML une saisie utilisateur non maîtrisée : risque d'injection de code (XSS). Pour afficher du texte, préférer textContent.

Autres accès

```
let formulaire = document.forms["inscription"];
let premiereImage = document.images[0];
let cartes = document.querySelectorAll(".carte");
for (let carte of cartes) {
  carte.classList.add("visible");
}
```

À retenir

Une collection DOM n'est pas nécessairement un tableau Array. Les noms des propriétés sont sensibles à la casse. Charger le script avec defer pour que les éléments HTML existent lors de leur sélection.

DOM : lire et contrôler un formulaire

Ouvrir cette partie sur STI v2.0 → | QR de lecture en bas de page

Propriétés essentielles

• input.value, textarea.value et select.value : valeur textuelle du champ.

• checkbox.checked et radio.checked : état booléen.

• select.options : collection des options ; options.length : taille.

• select.selectedIndex : indice de l’option sélectionnée, ou -1.

• option.selected, option.text et option.value : état, libellé et valeur.

```
<form id="calcul">
  <label for="note">Note</label>
  <input id="note" name="note" type="number"
         min="0" max="20" step="0.5" required>
  <label><input id="accord" type="checkbox" required>
    Je confirme la saisie</label>
  <button type="submit">Vérifier</button>
</form>
<p id="resultat"></p>

let formulaire = document.getElementById("calcul");
formulaire.addEventListener("submit", function (event) {
  event.preventDefault();
  let champ = document.getElementById("note");
  let accord = document.getElementById("accord").checked;
  let note = Number(champ.value);
  let sortie = document.getElementById("resultat");
  if (champ.value === "" || !Number.isFinite(note)
      || note < 0 || note > 20 || !accord) {
    sortie.textContent = "Vérifier les données.";
    return;
  }
  sortie.textContent = note >= 10 ? "Admis" : "Non admis";
});
```

Modifier une liste existante

```
let liste = document.getElementById("classe");
liste.add(new Option("4ème SI 3", "4SI3"));
if (liste.options.length > 0) { liste.remove(0); }
```

Pour les radios, document.querySelector('input[name="groupe"]:checked') renvoie le bouton sélectionné, ou null. Tester null avant d’accéder à sa valeur. Le contrôle serveur reste indispensable.

Fonctions, portée et événements

Ouvrir cette partie sur STI v2.0 → | QR de lecture en bas de page

Définir et appeler une fonction

```
function doubler(nombre) {
  let resultat = nombre * 2;
  return resultat;
}
let valeur = doubler(7);
console.log(valeur); // 14
```

Un paramètre est un nom dans la définition ; un argument est une valeur transmise lors de l'appel. return termine la fonction et renvoie une valeur. Sans retour explicite, le résultat est undefined.

Portée

```
let message = "Message extérieur";
function afficher() {
  let message = "Message local";
  console.log(message);
}
afficher();
console.log(message); // Message extérieur
```

Les arguments sont transmis par valeur. Pour un objet, la valeur transmise est une référence : modifier ses propriétés peut modifier l'objet partagé, mais réaffecter le paramètre ne réaffecte pas la variable de l'appelant.

Associer un événement

```
<button id="bouton" type="button">Cliquer</button>
<p id="sortie"></p>

function reagir(event) {
  document.getElementById("sortie").textContent = "Bonjour";
  console.log(event.currentTarget.id);
}
let bouton = document.getElementById("bouton");
bouton.addEventListener("click", reagir);
// Pour retirer cet écouteur :
// bouton.removeEventListener("click", reagir);
```

Méthodes et événements usuels

onclick = reagir associe un gestionnaire, remplacé par une affectation ultérieure. addEventListener autorise plusieurs écouteurs et est recommandé. Ne pas écrire reagir() lors de l'association : cela appellerait immédiatement la fonction.

Événements courants : click, input, change, submit, focus et blur. event.target désigne la cible initiale ; event.currentTarget l'élément portant l'écouteur. Éviter les chaînes de code dans les attributs d'événements.

Nombres, chaînes et méthodes usuelles

Ouvrir cette partie sur STI v2.0 → | QR de lecture en bas de page

Nombres et Math

```
let nombre = Number("123");
console.log(nombre.toString());   // "123"
console.log((6).toString(2));      // "110"
console.log(Number.isNaN(NaN));    // true
console.log(Math.floor(4.8));      // 4
console.log(Math.round(4.8));      // 5
console.log(Math.abs(-7));        // 7
```

Number utilise généralement une représentation IEEE 754 en double précision. Certains décimaux ne sont pas représentés exactement. Éviter new Number() pour un simple nombre : il crée un objet dont valueOf() extrait la valeur primitive.

Chaînes

```
let texte = "  Bonjour Tunis  ";
let propre = texte.trim();
console.log(propre.length);             // 13
console.log(propre.charAt(0));          // "B"
console.log(propre.indexOf("Tunis"));   // 8
console.log(propre.lastIndexOf("o"));   // 4
console.log(propre.substring(0, 7));    // "Bonjour"
console.log(propre.slice(-5));          // "Tunis"
console.log(propre.toUpperCase());      // "BONJOUR TUNIS"
console.log(propre.toLowerCase());      // "bonjour tunis"
console.log("Bonjour ".concat("Amal")); // "Bonjour Amal"
console.log(propre.split(" "));         // ["Bonjour", "Tunis"]
```

indexOf et lastIndexOf renvoient -1 en l'absence de correspondance. substring utilise un indice de fin exclu et échange les bornes si nécessaire. slice accepte des indices négatifs. L'ancienne méthode substr est à éviter dans de nouveaux exemples.

length mesure des unités de code UTF-16, pas toujours des caractères visibles : certains symboles comptent pour deux unités ou davantage. Les chaînes sont immuables : les méthodes renvoient de nouvelles valeurs.

Pourquoi éviter eval ?

eval exécute une chaîne comme du code. Il ne faut pas l'utiliser pour calculer une expression saisie par l'utilisateur. Employer les opérateurs et les conversions explicites.

Application

Nettoyer un nom avec trim, le convertir en majuscules et vérifier qu'il contient au moins deux caractères.

Tableaux et parcours

Ouvrir cette partie sur STI v2.0 → | QR de lecture en bas de page

Créer, lire et modifier un tableau

```
let fruits = ["Pomme", "Banane"];
console.log(fruits[0]);                 // Pomme
console.log(fruits.length);             // 2
fruits.push("Orange");                  // ajout à la fin
let dernier = fruits.pop();             // retire Orange
fruits.unshift("Fraise");               // ajout au début
let premier = fruits.shift();           // retire Fraise
console.log(fruits.join(", "));         // Pomme, Banane
console.log(fruits.toString());         // Pomme,Banane
```

Les indices commencent à zéro. Pour un tableau dense, le dernier indice est length - 1. Un indice absent renvoie undefined. Dans un tableau creux, length peut dépasser le nombre d'éléments réellement présents.

Parcourir

```
for (let i = 0; i < fruits.length; i++) {
  console.log(i, fruits[i]);
}
for (let fruit of fruits) {
  console.log(fruit);
}
fruits.forEach(function (fruit, indice) {
  console.log(indice, fruit);
});
```

for...of parcourt les valeurs. forEach appelle une fonction pour chaque élément présent ; break ne permet pas de quitter ce parcours. Utiliser une boucle classique si un arrêt anticipé est nécessaire.

Exemple de traitement

```
let notes = [12, 15, 9, 16];
let somme = 0;
for (let note of notes) { somme += note; }
let moyenne = notes.length > 0 ? somme / notes.length : null;
console.log(moyenne); // 13
notes.sort(function (a, b) { return a - b; });
console.log(notes);   // [9, 12, 15, 16]
```

sort sans fonction compare les représentations textuelles. Pour trier des nombres dans l'ordre croissant, fournir la fonction de comparaison a - b.

Application

Afficher la plus grande note et compter les notes supérieures ou égales à 10. Prévoir le cas d'un tableau vide.

Dates, temporisation et horloge numérique

Ouvrir cette partie sur STI v2.0 → | QR de lecture en bas de page

Objet Date

```
let maintenant = new Date();
let exemple = new Date(2026, 9, 21, 12, 30, 0);
console.log(exemple.getFullYear()); // 2026
console.log(exemple.getMonth());    // 9 : octobre
console.log(exemple.getDate());     // 21 : jour du mois
```

getDay renvoie le jour de la semaine de 0 (dimanche) à 6. getHours, getMinutes, getSeconds et getMilliseconds renvoient respectivement 0–23, 0–59, 0–59 et 0–999. Ces méthodes lisent l'heure locale du navigateur.

getTime() renvoie les millisecondes depuis le 1er janvier 1970 à 00:00:00 UTC ; setTime(ms) modifie la date. En PHP, un timestamp Unix est exprimé en secondes : ne pas confondre les unités.

TP : horloge

```
<!DOCTYPE html>
<html lang="fr">
<head>
  <meta charset="utf-8">
  <title>Horloge</title>
</head>
<body>
  <span id="horloge" aria-label="Heure actuelle"></span>
  <script>
    function afficherHorloge() {
      let date = new Date();
      let h = String(date.getHours()).padStart(2, "0");
      let m = String(date.getMinutes()).padStart(2, "0");
      let s = String(date.getSeconds()).padStart(2, "0");
      document.getElementById("horloge").textContent =
        h + ":" + m + ":" + s;
    }
    afficherHorloge();
    let minuterie = setInterval(afficherHorloge, 1000);
    // clearInterval(minuterie) permet l'arrêt.
  </script>
</body>
</html>
```

setTimeout(fonction, délai) prévoit un appel unique ; setInterval(fonction, délai) prévoit des appels répétés. Le délai est en millisecondes et n'est pas une garantie d'exécution exacte. Passer une fonction plutôt qu'une chaîne de code.

Compléments : Math, chaînes et sorties

Ouvrir cette partie sur STI v2.0 → | QR de lecture en bas de page

Math et nombres

```
console.log(Math.ceil(4.2));       // 5
console.log(Math.trunc(-4.8));     // -4
console.log(Math.sqrt(81));        // 9
console.log(Math.pow(2, 3));       // 8
console.log(Math.min(5, 2, 9));    // 2
console.log(Math.max(5, 2, 9));    // 9
console.log((12.345).toFixed(2));  // "12.35", une chaîne
console.log(isNaN("abc"));       // true : conversion préalable
console.log(Number.isNaN("abc")); // false : pas de conversion
```

Math.floor(-4.8) vaut -5, tandis que Math.trunc(-4.8) vaut -4. Les opérations décimales restent sujettes à la représentation binaire des nombres.

Remplacer et rechercher

```
let texte = "HTML et HTML";
console.log(texte.replace("HTML", "CSS")); // CSS et HTML
console.log(texte.replaceAll("HTML", "CSS")); // CSS et CSS
console.log(texte.includes("HTML")); // true
console.log("abc".startsWith("a")); // true
```

Les méthodes renvoient de nouvelles chaînes. La comparaison est généralement sensible à la casse.

Sorties : choisir le bon outil

console.log facilite le débogage. alert interrompt l’interaction jusqu’à sa fermeture. textContent affiche du texte ; innerHTML interprète du HTML. document.write peut remplacer le document lorsqu’il est appelé après son chargement : éviter son emploi dans les nouveaux exemples.

Ordre des opérations

```
console.log(1 + 2 + "3"); // "33"
console.log("1" + 2 + 3); // "123"
console.log("Total : " + (2 + 3)); // "Total : 5"
```

Application

Afficher une moyenne arrondie à deux décimales sans confondre une valeur numérique et sa représentation textuelle. Vérifier une saisie avec Number.isFinite après conversion explicite.

SCANNER LA FICHE
ou cliquer sur le QR code

Médias, styles et dates de formulaire

Ouvrir cette partie sur STI v2.0 → | QR de lecture en bas de page

Contrôler un média existant

```
<video id="film" controls src="demo.mp4"></video>
<button id="lecture" type="button">Lire</button>
<button id="pause" type="button">Pause</button>
<p id="message"></p>

let film = document.getElementById("film");
document.getElementById("lecture").addEventListener("click",
  function () {
    film.play().catch(function () {
      document.getElementById("message").textContent =
        "Lecture impossible : vérifier le fichier et les autorisations.";
    });
  });
document.getElementById("pause").addEventListener("click",
  function () { film.pause(); });
```

play() renvoie une promesse : la lecture peut être refusée par le navigateur ou échouer. Un clic utilisateur est préférable à un démarrage automatique non demandé.

Modifier la présentation

```
let element = document.getElementById("message");
element.style.backgroundColor = "#edf5f6";
element.classList.toggle("important");
```

Une propriété CSS avec tiret devient généralement une propriété JavaScript en camelCase. Préférer classList pour des ensembles de styles réutilisables.

Date non future

```
function dateNonFuture(valeur) {
  if (!/^\d{4}-\d{2}-\d{2}$/.test(valeur)) { return false; }
  let morceaux = valeur.split("-").map(Number);
  let a = morceaux[0], m = morceaux[1], j = morceaux[2];
  let date = new Date(a, m - 1, j);
  let limite = new Date();
  limite.setHours(0, 0, 0, 0);
  return date.getFullYear() === a && date.getMonth() === m - 1
    && date.getDate() === j && date <= limite;
}
```

Le constructeur normalise les dépassements : comparer les composantes permet de rejeter une date comme le 31 février. Vérifier la même règle sur le serveur, dont le fuseau peut différer de celui du navigateur.

Notions relationnelles et contraintes

Ouvrir cette partie sur STI v2.0 → | QR de lecture en bas de page

Vocabulaire

Une base de données est un ensemble organisé et persistant de données. Un SGBD permet de définir les structures, manipuler les données, gérer les accès concurrents et contrôler leur intégrité et leur sécurité. MySQL est un SGBD relationnel ; SQL est le langage utilisé pour dialoguer avec lui.

• Entité : objet du monde réel, par exemple un élève ou une classe.

• Attribut : propriété d'une entité, par exemple son nom.

• Table : ensemble de lignes décrites par des colonnes.

• Ligne, enregistrement ou n-uplet : occurrence d'une entité ou d'une association.

• Colonne ou champ : propriété ayant un type et des contraintes.

Clés et intégrité

Une clé primaire identifie chaque ligne ; elle est unique et non nulle. Elle peut être composée de plusieurs colonnes. Une clé étrangère référence une clé d'une autre table et assure la cohérence des liens.

NOT NULL impose une valeur ; UNIQUE impose l'unicité ; CHECK impose une condition ; FOREIGN KEY définit une référence. Une colonne nullable peut contenir NULL, qui signifie une information absente ou inconnue, et non une chaîne vide ou zéro.

Relation un à plusieurs

CLASSE(id_classe, nom_classe)

ELEVE(num_inscription, nom, prenom, date_naissance, adresse, id_classe#)

Une classe contient plusieurs élèves ; chaque élève appartient à une classe. La clé id_classe est ajoutée à ELEVE comme clé étrangère. Le symbole # est une convention de description, pas une syntaxe de nom de colonne SQL.

Relation plusieurs à plusieurs

ATHLETE(anum, anom, aage)

COURSE(cnum, cnom)

PARTICIPATION(cnum#, anum#)

Un athlète participe à plusieurs courses et une course accueille plusieurs athlètes. PARTICIPATION porte deux clés étrangères et une seule clé primaire composée de (cnum, anum).

Intérêts d'une base

Centraliser les informations, limiter les redondances inutiles, partager les données et assurer leur cohérence. La confidentialité dépend des droits d'accès ; elle n'est pas automatiquement garantie par le seul stockage en base.

Application

Identifier les clés primaires et étrangères d'un système de location de véhicules. Expliquer pourquoi un nom de client ne constitue pas une bonne clé primaire.

SQL : commandes et types MySQL

Ouvrir cette partie sur STI v2.0 → | QR de lecture en bas de page

Familles de commandes

• Définition des données : CREATE, ALTER, DROP.

• Manipulation des données : INSERT, UPDATE, DELETE ; interrogation : SELECT.

• Contrôle des accès : GRANT et REVOKE, étudiés ici uniquement comme repères.

Les exemples suivants ciblent MySQL 8.0.16 ou ultérieur, avec le moteur InnoDB. À partir de cette version, les contraintes CHECK sont effectivement appliquées. Des différences peuvent exister avec MariaDB et d'autres SGBD.

```
CREATE DATABASE gestion_location
  CHARACTER SET utf8mb4;
USE gestion_location;
SHOW DATABASES;
SHOW TABLES;
```

Types numériques

• TINYINT : entier signé de -128 à 127 ; UNSIGNED de 0 à 255.

• INT : entier signé de -2147483648 à 2147483647 ; UNSIGNED de 0 à 4294967295.

• DECIMAL(p,s) : décimal exact, p chiffres au total dont s après la virgule. DECIMAL(8,3) autorise cinq chiffres avant et trois après le point.

• NUMERIC est un synonyme de DECIMAL dans MySQL.

• FLOAT et DOUBLE sont approximatifs : préférer DECIMAL pour les montants monétaires.

Types textuels et binaires

• CHAR(n) : chaîne de longueur fixe ; VARCHAR(n) : longueur variable maximale n, limitée aussi par l'encodage et la taille maximale d'une ligne.

• TEXT : texte ; BLOB : données binaires. Leur taille est limitée en octets, et non toujours à 65535 caractères.

• La sensibilité à la casse d'un texte dépend de sa collation ; TEXT ne signifie pas « ASCII ».

• Téléphone et CIN sont des chaînes : un zéro initial doit être conservé.

Types temporels

DATE : 2026-10-21 ; TIME : 12:30:00 ; DATETIME : 2026-10-21 12:30:00 ; YEAR : 2026.

TIMESTAMP applique des conversions liées au fuseau horaire de la session ; DATETIME stocke une date et une heure civiles sans cette conversion automatique.

Conventions

Utiliser des identifiants sans espaces et conserver exactement la même casse pour les noms de tables. Les dates littérales sont écrites 'AAAA-MM-JJ'. Les chaînes utilisent des apostrophes droites.

SCANNER LA FICHE
ou cliquer sur le QR code

Création des tables : gestion des locations

Ouvrir cette partie sur STI v2.0 → | QR de lecture en bas de page

Schéma cohérent

Créer les tables parentes avant la table qui les référence. La clé de location inclut la date pour permettre plusieurs locations d'un même véhicule par un même client à des dates différentes.

```
CREATE TABLE client (
  ncin CHAR(8) PRIMARY KEY,
  nom VARCHAR(50) NOT NULL,
  prenom VARCHAR(50) NOT NULL,
  adresse VARCHAR(100),
  ville VARCHAR(50) DEFAULT 'Tunis'
) ENGINE=InnoDB;

CREATE TABLE vehicule (
  immat VARCHAR(20) PRIMARY KEY,
  marque VARCHAR(30) NOT NULL,
  modele VARCHAR(30) NOT NULL,
  annee_acq YEAR NOT NULL,
  CONSTRAINT ck_annee CHECK (annee_acq >= 1990)
) ENGINE=InnoDB;

CREATE TABLE location (
  immat VARCHAR(20) NOT NULL,
  ncin CHAR(8) NOT NULL,
  date_loc DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  duree_loc INT NOT NULL,
  cout_loc DECIMAL(8,3) NOT NULL,
  PRIMARY KEY (immat, ncin, date_loc),
  CONSTRAINT fk_location_vehicule FOREIGN KEY (immat)
    REFERENCES vehicule(immat) ON DELETE RESTRICT,
  CONSTRAINT fk_location_client FOREIGN KEY (ncin)
    REFERENCES client(ncin) ON DELETE RESTRICT,
  CONSTRAINT ck_duree CHECK (duree_loc > 0),
  CONSTRAINT ck_cout CHECK (cout_loc > 0)
) ENGINE=InnoDB;
```

Intégrité référentielle

RESTRICT refuse la suppression d'un parent encore référencé. CASCADE supprime automatiquement ses enfants : à choisir seulement si cela correspond à la règle de gestion. Pour conserver un historique de locations, RESTRICT est plus prudent.

Dates et contraintes

Ne pas écrire CHECK(date_loc <= NOW()) : MySQL n'autorise pas cette fonction non déterministe dans un CHECK. Vérifier cette règle dans le traitement applicatif ou un déclencheur adapté. DEFAULT CURRENT_TIMESTAMP n'est pas réservé au seul type TIMESTAMP ; il est aussi utilisable avec DATETIME.

Modifier et supprimer une structure

Ouvrir cette partie sur STI v2.0 → | QR de lecture en bas de page

Exemples indépendants à tester sur une copie

```
ALTER TABLE client ADD COLUMN telephone VARCHAR(8);
ALTER TABLE client MODIFY COLUMN telephone VARCHAR(20);
ALTER TABLE client ALTER COLUMN ville SET DEFAULT 'Tunis';
ALTER TABLE client DROP COLUMN adresse;
ALTER TABLE vehicule ADD CONSTRAINT ck_annee_max
  CHECK (annee_acq <= 2026);
ALTER TABLE vehicule DROP CHECK ck_annee_max;
DESCRIBE client;
SHOW CREATE TABLE location;
```

Un numéro de téléphone reste textuel. Lors d'un MODIFY COLUMN, préciser les propriétés à conserver, notamment NOT NULL et DEFAULT, car certaines peuvent être perdues si on les omet.

Clé primaire composée

Pour une table location ancienne dont la clé était seulement (immat, ncin), la correction est :

```
ALTER TABLE location
  DROP PRIMARY KEY,
  ADD PRIMARY KEY (immat, ncin, date_loc);
```

La virgule sépare les deux modifications. Dans le schéma de la fiche précédente, la clé est déjà correcte : il n'est pas nécessaire de la modifier.

Contraintes et renommage

```
ALTER TABLE location DROP FOREIGN KEY fk_location_client;
ALTER TABLE vehicule RENAME TO voiture;
```

Ne pas supprimer une clé primaire référencée sans analyser les clés étrangères et les index associés. DISABLE KEYS et ENABLE KEYS ne désactivent pas les contraintes de clé primaire ou de clé étrangère InnoDB ; ce ne sont pas des commandes de suspension de l'intégrité.

Suppression : prudence

```
-- À exécuter seulement sur une base de test sauvegardée.
DROP TABLE location;
DROP TABLE vehicule;
DROP TABLE client;
-- DROP DATABASE gestion_location;
```

Supprimer les tables enfants avant les parentes. Ces commandes supposent que vehicule n'a pas été renommée. DROP détruit une structure ; DELETE supprime des lignes en conservant la table.

Application

Ajouter une colonne email au client, puis vérifier la nouvelle structure. Ne pas exécuter toutes les commandes de cette fiche à la suite sur la base servant aux autres exercices.

SCANNER LA FICHE
ou cliquer sur le QR code

Insérer, modifier et supprimer des lignes

Ouvrir cette partie sur STI v2.0 → | QR de lecture en bas de page

INSERT : préciser les colonnes

```
INSERT INTO client (ncin, nom, prenom, ville)
VALUES ('01234567', 'Tounsi', 'Wael', 'Tunis');
INSERT INTO vehicule (immat, marque, modele, annee_acq)
VALUES ('123TU4567', 'Renault', 'Clio', 2026);
INSERT INTO location (immat, ncin, date_loc, duree_loc, cout_loc)
VALUES ('123TU4567', '01234567', '2026-10-21 09:00:00',
        3, 180.000);
```

Les valeurs doivent respecter les types, NOT NULL, l'unicité, les CHECK et les clés étrangères. Insérer les parents avant les enfants. Une chaîne numérique comme la CIN garde ses apostrophes pour préserver sa forme.

UPDATE : attention à WHERE

```
UPDATE client
SET ville = 'Sfax', telephone = '20123456'
WHERE ncin = '01234567';

UPDATE location
SET cout_loc = ROUND(cout_loc * 1.18, 3)
WHERE date_loc >= '2026-10-01'
  AND date_loc < '2026-11-01';
```

Le premier exemple suppose la colonne telephone ajoutée dans la fiche précédente. Le second augmente de 18 % le coût des locations d'octobre 2026. Sans WHERE, UPDATE modifierait toutes les lignes.

DELETE

```
DELETE FROM location
WHERE immat = '123TU4567'
  AND ncin = '01234567'
  AND date_loc = '2026-10-21 09:00:00';
```

Sans WHERE, DELETE FROM location supprime toutes les lignes de location. La suppression d'un parent référencé est refusée avec RESTRICT, ou propagée avec CASCADE si cette action a été définie.

Méthode de travail

1. Écrire un SELECT avec la même condition pour vérifier les lignes visées.
2. Vérifier la sauvegarde et, si nécessaire, utiliser une transaction.
3. Exécuter la modification et contrôler le nombre de lignes affectées.

Exercice

Changer la durée d'une location précisément identifiée, puis supprimer uniquement cette location. Expliquer pourquoi une condition portant seulement sur la CIN peut viser plusieurs lignes.

Base commerciale : schéma des exercices

Ouvrir cette partie sur STI v2.0 → | QR de lecture en bas de page

Un second domaine, indépendant des locations

Les exemples suivants utilisent une base de gestion commerciale. Chaque ligne de ventes représente la vente d'un article ; plusieurs lignes peuvent concerner le même client le même jour.

```
CREATE DATABASE commerce CHARACTER SET utf8mb4;
USE commerce;
CREATE TABLE fournisseurs (
  num_frs INT PRIMARY KEY,
  nom_frs VARCHAR(50) NOT NULL,
  tel_frs VARCHAR(20)
) ENGINE=InnoDB;
CREATE TABLE clients (
  cod_cli CHAR(4) PRIMARY KEY,
  nom_cli VARCHAR(50) NOT NULL,
  prn_cli VARCHAR(50) NOT NULL,
  nat_cli VARCHAR(30)
) ENGINE=InnoDB;
CREATE TABLE articles (
  num_art INT PRIMARY KEY,
  des_art VARCHAR(80) NOT NULL,
  col_art VARCHAR(30),
  pv_art DECIMAL(10,3) NOT NULL CHECK (pv_art > 0),
  pa_art DECIMAL(10,3) NOT NULL CHECK (pa_art > 0),
  num_frs INT NOT NULL,
  FOREIGN KEY (num_frs) REFERENCES fournisseurs(num_frs)
) ENGINE=InnoDB;
CREATE TABLE ventes (
  id_vnt INT AUTO_INCREMENT PRIMARY KEY,
  cod_cli CHAR(4) NOT NULL,
  num_art INT NOT NULL,
  dat_vnt DATETIME NOT NULL,
  qte_vnt INT NOT NULL CHECK (qte_vnt > 0),
  prix_unitaire DECIMAL(10,3) NOT NULL CHECK (prix_unitaire > 0),
  FOREIGN KEY (cod_cli) REFERENCES clients(cod_cli),
  FOREIGN KEY (num_art) REFERENCES articles(num_art)
) ENGINE=InnoDB;
CREATE TABLE magasins (
  num_mag INT PRIMARY KEY,
  loc_mag VARCHAR(50) NOT NULL
) ENGINE=InnoDB;
```

prix_unitaire conserve le prix réellement appliqué au moment de la vente. Un total historique doit utiliser ce prix, et non le prix courant de articles. Ce schéma simplifié ne sépare pas encore l'en-tête d'une facture et ses lignes.

SELECT, filtres et expressions

Ouvrir cette partie sur STI v2.0 → | QR de lecture en bas de page

Projection, sélection et DISTINCT

```
SELECT nom_cli, prn_cli FROM clients;
SELECT * FROM clients WHERE nat_cli = 'Tunisienne';
SELECT DISTINCT nat_cli FROM clients;
SELECT num_art, des_art, pv_art - pa_art AS marge
FROM articles;
SELECT CONCAT(prn_cli, ' ', nom_cli) AS nom_complet
FROM clients;
```

Une projection choisit des colonnes ; une sélection filtre des lignes. DISTINCT élimine les doublons du résultat. En MySQL, utiliser CONCAT pour concaténer : || est généralement un OU logique, sauf mode SQL particulier.

Conditions

```
SELECT * FROM articles WHERE pv_art BETWEEN 10 AND 70;
SELECT * FROM magasins WHERE loc_mag IN ('Sfax', 'Gabès');
SELECT * FROM magasins WHERE loc_mag NOT IN ('Sfax', 'Gabès');
SELECT * FROM articles
WHERE des_art LIKE 'Ch%' AND pv_art > 70;
SELECT * FROM fournisseurs WHERE nom_frs LIKE 'Ka____';
SELECT * FROM clients WHERE nat_cli IS NULL;
```

BETWEEN inclut ses bornes. IN teste l’appartenance à une liste ; LIKE recherche un motif : % correspond à zéro ou plusieurs caractères, _ à un seul caractère. 'Ka____' commence par Ka et compte six caractères.

Les opérateurs usuels sont =, <>, !=, <, <=, >, >= ; !< et !> ne sont pas des opérateurs MySQL valides. Combiner les conditions avec AND, OR et NOT et ajouter des parenthèses pour clarifier l’intention.

Dates : intervalle semi-ouvert

```
SELECT * FROM ventes
WHERE dat_vnt >= '2026-01-01'
  AND dat_vnt < '2026-01-09';
```

Cette condition couvre du 1er au 8 janvier inclus, même si dat_vnt contient une heure. Une borne supérieure '2026-01-08' ne couvrirait que minuit de cette dernière journée.

NULL et casse

Ne pas écrire = NULL : utiliser IS NULL ou IS NOT NULL. Les comparaisons de texte et LIKE dépendent de la collation ; UPPER peut normaliser la casse, mais ne corrige pas des libellés incohérents.

Agrégats, tri et sous-requêtes

Ouvrir cette partie sur STI v2.0 → | QR de lecture en bas de page

Fonctions d’agrégation

COUNT(*) compte les lignes ; COUNT(colonne) compte les valeurs non nulles ; COUNT(DISTINCT colonne) compte les valeurs distinctes non nulles. SUM, AVG, MIN et MAX calculent somme, moyenne, minimum et maximum en ignorant les valeurs NULL.

```
SELECT COUNT(*) AS nb_fournisseurs FROM fournisseurs;
SELECT COUNT(*) AS nb_articles FROM articles WHERE num_frs = 777;
SELECT SUM(pa_art) AS somme, AVG(pa_art) AS moyenne,
       MIN(pv_art) AS prix_min, MAX(pv_art) AS prix_max
FROM articles;
SELECT COUNT(DISTINCT cod_cli) AS nb_clients
FROM ventes
WHERE dat_vnt >= '2026-01-01' AND dat_vnt < '2026-01-09';
```

SUM(pa_art)/COUNT(*) ne reproduit AVG(pa_art) que si pa_art ne contient pas de NULL. Une somme de prix unitaires n’est pas une valeur de stock : celle-ci nécessiterait aussi des quantités.

Sous-requêtes

```
SELECT des_art FROM articles
WHERE pv_art = (SELECT MAX(pv_art) FROM articles);
SELECT num_art FROM articles
WHERE pv_art = (SELECT MIN(pv_art) FROM articles);
SELECT num_frs FROM articles
WHERE pa_art = (SELECT MAX(pa_art) FROM articles);
SELECT des_art FROM articles
WHERE pv_art > (SELECT AVG(pv_art) FROM articles);
```

Ces requêtes conservent les ex æquo. Un agrégat ne s’utilise pas directement dans le WHERE du même niveau de requête ; il peut apparaître dans une sous-requête, comme ici.

Tri

```
SELECT * FROM clients ORDER BY nom_cli ASC, prn_cli ASC;
SELECT * FROM clients ORDER BY prn_cli DESC;
SELECT * FROM articles ORDER BY pa_art DESC, pv_art ASC;
```

ASC est l’ordre croissant par défaut ; DESC l’ordre décroissant. Sans ORDER BY, l’ordre des résultats n’est pas garanti.

Application

Afficher les articles dont le prix dépasse la moyenne, du plus cher au moins cher. Compter les clients distincts ayant effectué au moins une vente en octobre 2026.

GROUP BY, HAVING et jointures

Ouvrir cette partie sur STI v2.0 → | QR de lecture en bas de page

Regrouper puis filtrer les groupes

```
SELECT num_frs, COUNT(*) AS nb_articles
FROM articles GROUP BY num_frs;
SELECT DATE(dat_vnt) AS jour, SUM(qte_vnt) AS quantite_vendue
FROM ventes GROUP BY DATE(dat_vnt);
SELECT cod_cli, COUNT(*) AS nb_lignes_vente
FROM ventes GROUP BY cod_cli HAVING COUNT(*) <= 2;
SELECT num_art FROM ventes
GROUP BY num_art HAVING COUNT(*) > 2;
```

WHERE filtre les lignes avant le regroupement ; HAVING filtre les groupes après calcul. « Deux au plus » signifie <= 2, et non >= 2. Ce comptage concerne les lignes de vente, pas les unités ni les factures. Les clients sans vente ne figurent pas dans le regroupement de ventes.

Les colonnes sélectionnées hors agrégat doivent figurer dans GROUP BY, sauf dépendance fonctionnelle reconnue par le SGBD. Pour débuter, écrire explicitement toutes ces colonnes dans GROUP BY.

Jointure interne

```
SELECT a.num_art, f.nom_frs, f.tel_frs
FROM articles AS a
JOIN fournisseurs AS f ON f.num_frs = a.num_frs;
SELECT DISTINCT a.des_art, a.col_art
FROM articles AS a
JOIN ventes AS v ON v.num_art = a.num_art;
```

JOIN ... ON rend la liaison explicite. Les alias évitent les ambiguïtés ; DISTINCT évite ici de répéter un article vendu plusieurs fois.

Total des ventes par client

```
SELECT c.cod_cli, c.nom_cli,
       SUM(v.qte_vnt * v.prix_unitaire) AS total_ventes
FROM clients AS c
JOIN ventes AS v ON v.cod_cli = c.cod_cli
GROUP BY c.cod_cli, c.nom_cli
HAVING SUM(v.qte_vnt * v.prix_unitaire) > 1000;
```

Absence de correspondance

```
SELECT f.nom_frs, f.tel_frs FROM fournisseurs AS f
WHERE NOT EXISTS (
  SELECT 1 FROM articles AS a WHERE a.num_frs = f.num_frs
);
```

Pour les articles jamais vendus, appliquer la même construction entre articles et ventes. NOT EXISTS évite notamment les pièges de NOT IN lorsque la sous-requête peut contenir NULL.

Fonctions SQL sur les chaînes

Ouvrir cette partie sur STI v2.0 → | QR de lecture en bas de page

Fonctions MySQL usuelles

```
SELECT CONCAT('Amine', ' ', 'Benali') AS nom_complet;
SELECT UPPER('Tunis'), LOWER('HTML');
SELECT LENGTH('Tunis') AS octets,
       CHAR_LENGTH('Tunis') AS caracteres;
SELECT SUBSTRING('abcdefghij', 2, 3) AS extrait; -- bcd
SELECT LEFT('abcdefghij', 2) AS debut;          -- ab
SELECT RIGHT('abcdefghij', 3) AS fin;           -- hij
SELECT TRIM('  STI  ') AS propre;
SELECT REPLACE('HTML et HTML', 'HTML', 'CSS') AS resultat;
```

LENGTH mesure des octets ; CHAR_LENGTH mesure des caractères. Avec un encodage multioctet comme UTF-8, les résultats peuvent différer. En SQL, SUBSTRING utilise une position initiale 1, contrairement aux indices de chaîne JavaScript qui commencent à 0.

Appliquer aux données

```
USE commerce;
SELECT cod_cli, CONCAT(prn_cli, ' ', nom_cli) AS identite
FROM clients ORDER BY nom_cli, prn_cli;
SELECT num_art, LEFT(des_art, 10) AS libelle_court
FROM articles;
SELECT * FROM fournisseurs
WHERE CHAR_LENGTH(nom_frs) = 6 AND nom_frs LIKE 'Ka%';
UPDATE clients SET nat_cli = UPPER(nat_cli);
```

L’UPDATE touche toutes les lignes : le tester d’abord avec SELECT. CONCAT retourne NULL si l’un de ses arguments est NULL ; COALESCE permet de remplacer une valeur absente.

```
SELECT CONCAT(prn_cli, ' ', nom_cli, ' - ',
              COALESCE(nat_cli, 'Non renseignée')) AS fiche
FROM clients;
```

Application

Afficher les trois premières lettres du nom de chaque fournisseur. Rechercher les noms qui contiennent six caractères et commencent par Ka. Comparer les longueurs en octets et en caractères d’un texte contenant « é ».

SCANNER LA FICHE
ou cliquer sur le QR code

Fonctions SQL sur les dates et heures

Ouvrir cette partie sur STI v2.0 → | QR de lecture en bas de page

Lire et extraire

```
SELECT NOW() AS instant, CURRENT_DATE() AS jour;
SELECT YEAR('2026-10-21') AS annee,
       MONTH('2026-10-21') AS mois,
       DAY('2026-10-21') AS jour;
SELECT HOUR('12:30:45'), MINUTE('12:30:45'), SECOND('12:30:45');
SELECT DATE('2026-10-21 12:30:00') AS date_seule;
SELECT TIMESTAMP('2026-10-21', '12:30:00') AS instant;
```

NOW dépend du fuseau horaire de la session MySQL. Il n’est pas réglé par date_default_timezone_set en PHP. TIMESTAMP() est ici une fonction de construction, à distinguer du type de colonne TIMESTAMP et d’un timestamp Unix en secondes.

Ajouter, retrancher et comparer

```
SELECT DATE_ADD('2026-08-25', INTERVAL 3 MONTH); -- 2026-11-25
SELECT DATE_SUB('2026-10-21', INTERVAL 7 DAY);   -- 2026-10-14
SELECT DATEDIFF('2026-08-25', '2026-08-15');     -- 10
SELECT TIMESTAMPDIFF(YEAR, '2008-03-14', '2026-10-21'); -- 18
```

DATEDIFF(a,b) calcule a moins b en jours et ignore les heures. TIMESTAMPDIFF(unite,debut,fin) mesure des unités entières entre deux instants ; l’ordre des arguments diffère de celui de DATEDIFF.

Rechercher une période

```
SELECT * FROM ventes
WHERE dat_vnt >= '2026-10-01' AND dat_vnt < '2026-11-01';
SELECT DATE(dat_vnt) AS jour, SUM(qte_vnt) AS quantite
FROM ventes GROUP BY DATE(dat_vnt) ORDER BY jour;
```

Pour un filtre de période, comparer la colonne directement à des bornes peut permettre un meilleur usage des index que YEAR(dat_vnt) ou MONTH(dat_vnt). Pour un regroupement journalier, DATE(dat_vnt) exprime explicitement l’intention.

Application

Calculer une date de retour à partir de date_loc et duree_loc dans gestion_location. Compter les clients distincts ayant acheté pendant les sept dernières journées civiles complètes.

Jointures STI, mises à jour croisées et LCD

Ouvrir cette partie sur STI v2.0 → | QR de lecture en bas de page

Deux écritures d’une jointure interne

```
SELECT a.num_art, f.nom_frs
FROM articles a, fournisseurs f
WHERE a.num_frs = f.num_frs;

SELECT a.num_art, f.nom_frs
FROM articles a
JOIN fournisseurs f ON a.num_frs = f.num_frs;
```

La première écriture correspond à la présentation STI de la plateforme. Les deux sont équivalentes ici. Oublier la liaison avec une liste de tables séparées par des virgules produit un produit cartésien.

UPDATE utilisant deux tables

Exemple indépendant : appliquer des tarifs transmis par les fournisseurs à des articles sélectionnés. Créer la table de correspondance avant la mise à jour.

```
CREATE TABLE nouveaux_prix (
  num_art INT PRIMARY KEY,
  nouveau_prix DECIMAL(10,3) NOT NULL CHECK (nouveau_prix > 0)
);
INSERT INTO nouveaux_prix VALUES (501, 18.750);
SELECT a.num_art, a.pv_art, n.nouveau_prix
FROM articles a, nouveaux_prix n WHERE a.num_art = n.num_art;
UPDATE articles a, nouveaux_prix n
SET a.pv_art = n.nouveau_prix
WHERE a.num_art = n.num_art;
```

La clé primaire de nouveaux_prix évite plusieurs prix candidats pour un même article. Vérifier les correspondances avec SELECT et sauvegarder avant modification.

Contrôle des accès : repères

```
-- À exécuter uniquement avec un compte administrateur adapté.
CREATE USER 'lecteur_sti'@'localhost' IDENTIFIED BY 'A_remplacer_2026!';
GRANT SELECT ON commerce.* TO 'lecteur_sti'@'localhost';
REVOKE SELECT ON commerce.* FROM 'lecteur_sti'@'localhost';
```

Ces commandes sont des exemples MySQL 8 ; ne pas employer ce mot de passe de démonstration en production. Le compte d’une application doit recevoir uniquement les droits nécessaires. Une révocation de droit n’efface ni la table ni ses données.

Application

Comparer les deux écritures de jointure puis préparer une mise à jour croisée sur une copie de la base. Distinguer les commandes LDD, LMD et LCD.

Principe client-serveur et environnement

Ouvrir cette partie sur STI v2.0 → | QR de lecture en bas de page

Une page dynamique

Le navigateur envoie une requête HTTP au serveur. Le serveur exécute le script PHP, peut interroger MySQL, puis renvoie une réponse, souvent HTML. Le navigateur reçoit cette réponse, pas le code source PHP lorsque le serveur est correctement configuré.

Navigateur → requête HTTP → serveur Web + PHP → MySQL si nécessaire → réponse HTML → navigateur.

Le traitement côté serveur permet de centraliser les règles et l'accès aux données. Il ne garantit ni un téléchargement plus rapide dans tous les cas, ni l'absence de problèmes de compatibilité du HTML produit.

Environnement de travail

Utiliser une version PHP 8.x maintenue, un serveur Web configuré pour PHP et un serveur MySQL. Un environnement local intégré peut fournir ces composants. phpMyAdmin est une interface d'administration de MySQL, pas le serveur de base de données lui-même.

Enregistrer les scripts avec l'extension .php dans le dossier servi par le serveur. Ce dossier dépend de la configuration : il ne s'appelle pas nécessairement www. Ne pas ouvrir le fichier PHP directement avec file://.

Première page

```
<!DOCTYPE html>
<html lang="fr">
<head>
  <meta charset="utf-8">
  <title>Première page PHP</title>
</head>
<body>
  <h1><?php echo "Bienvenue au Lycée Rafèha"; ?></h1>
</body>
</html>
```

Syntaxe et commentaires

Commencer le code par <?php. Les instructions se terminent par un point-virgule ; // et /* … */ servent aux commentaires. Dans un fichier contenant exclusivement du PHP, on omet généralement la balise fermante pour éviter des sorties accidentelles.

PHP signifie « PHP: Hypertext Preprocessor ». Le langage est à typage dynamique et sensible à la casse pour les noms de variables.

Convention du cours

Les accès aux bases utilisent exclusivement l'API MySQLi procédurale, sous la forme mysqli_*. Le parcours des tableaux est effectué avec foreach ou des boucles classiques.

Variables, types, constantes et opérateurs

Ouvrir cette partie sur STI v2.0 → | QR de lecture en bas de page

Variables et constantes

```
<?php
$nom = "Amal";
$age = 18;
$moyenne = 15.5;
$admis = true;
$absence = null;
define("ETABLISSEMENT", "Lycée Rafèha");
echo ETABLISSEMENT . " : " . $nom;
```

Une variable commence par $, puis une lettre ou un soulignement, suivis éventuellement de chiffres. $nom et $Nom désignent deux variables différentes. Les constantes ne commencent pas par $.

Les types usuels sont int, float, string, bool, array et null. Une nouvelle affectation peut changer le type d'une variable non typée, mais les opérations et les paramètres restent soumis aux règles de type de PHP.

Fonctions sur les variables

```
<?php
$x = 2.3;
echo gettype($x);        // double (nom historique)
settype($x, "integer");
echo $x;                // 2
$present = isset($x);    // true
unset($x);
$present = isset($x);    // false
$valeur = null;
$present = isset($valeur); // false
```

isset retourne false si la variable n'existe pas ou vaut null. Un cast comme (int)$x effectue une conversion. Il ne remplace pas la validation d'une entrée utilisateur.

Opérateurs

+ - * / % ** : calcul ; . : concaténation ; = : affectation ; += et .= : mise à jour. == compare avec conversions éventuelles ; === compare valeur et type. Utiliser &&, || et ! pour les expressions logiques usuelles.

```
<?php
$prix = 25.5;
$quantite = 3;
$total = $prix * $quantite;
$message = "Total : " . $total . " DT";
echo $message;
```

Application

Définir une constante TAUX_TVA, calculer un prix TTC et afficher un message. Expliquer la différence entre une conversion de type et le contrôle de validité d'une saisie.

Formulaires : lecture, validation et affichage

Ouvrir cette partie sur STI v2.0 → | QR de lecture en bas de page

Traitement d’une requête POST

Le formulaire envoie les valeurs selon les attributs name. $_GET et $_POST sont des tableaux associatifs. Vérifier qu’une valeur existe, qu’elle est du type attendu et qu’elle respecte les règles du domaine.

```
<?php
if ($_SERVER["REQUEST_METHOD"] !== "POST") {
    http_response_code(405);
    exit("Méthode non autorisée.");
}
$nomBrut = $_POST["nom"] ?? null;
$emailBrut = $_POST["email"] ?? null;
if (!is_string($nomBrut) || !is_string($emailBrut)) {
    http_response_code(400);
    exit("Données manquantes ou invalides.");
}
$nom = trim($nomBrut);
$email = filter_var(trim($emailBrut), FILTER_VALIDATE_EMAIL);
if ($nom === "" || strlen($nom) > 200 || $email === false) {
    http_response_code(400);
    exit("Nom ou adresse électronique invalide.");
}
echo "Bonjour " . htmlspecialchars(
    $nom, ENT_QUOTES | ENT_SUBSTITUTE, "UTF-8"
);
```

La limite de 200 ci-dessus porte sur des octets, car strlen compte des octets. Pour une règle en nombre de caractères Unicode, utiliser mb_strlen avec l’extension mbstring.

Valider un entier

```
<?php
$ageBrut = $_POST["age"] ?? null;
if (!is_string($ageBrut)) { exit("Âge invalide."); }
$age = filter_var($ageBrut, FILTER_VALIDATE_INT, [
    "options" => ["min_range" => 0, "max_range" => 120]
]);
if ($age === false) { exit("Âge invalide."); }
```

Sécurité : trois problèmes distincts

• Validation : vérifier la signification de la valeur.

• Sortie HTML : utiliser htmlspecialchars pour le texte et les attributs ordinaires entre guillemets.

• Requête SQL : utiliser des paramètres liés, pas une concaténation de saisies.

Une modification sensible doit aussi vérifier l’autorisation de l’utilisateur et protéger contre les requêtes intersites (jeton CSRF). HTTPS protège le transport, sans remplacer ces contrôles.

Conditions, boucles et fonctions

Ouvrir cette partie sur STI v2.0 → | QR de lecture en bas de page

Conditions

```
<?php
$note = 14;
if ($note < 0 || $note > 20) {
    echo "Note invalide";
} elseif ($note >= 10) {
    echo "Admis";
} else {
    echo "Non admis";
}
```

switch fonctionne avec case, break et default. En PHP, switch utilise des comparaisons non strictes : ne pas lui attribuer le comportement strict du switch JavaScript.

Boucles

```
<?php
$somme = 0;
for ($i = 1; $i <= 10; $i++) {
    $somme += $i;
}
echo $somme; // 55
$n = 10;
$factorielle = 1;
while ($n > 0) {
    $factorielle *= $n;
    $n--;
}
$i = 0;
do {
    echo $i . "<br>";
    $i++;
} while ($i < 3);
```

Initialiser les accumulateurs avant la boucle. while teste avant le bloc ; do...while teste après et s’exécute au moins une fois.

Fonction et passage de paramètres

```
<?php
function calculerMoyenne($somme, $effectif) {
    if ($effectif <= 0) { return null; }
    return $somme / $effectif;
}
echo calculerMoyenne(42, 3); // 14
```

return termine l’appel et fournit une valeur. Sans retour explicite, une fonction renvoie null. Les paramètres sont transmis par valeur par défaut. Le préfixe & permet un passage par référence, à utiliser uniquement si la modification de la variable de l’appelant est voulue. Les variables d’une fonction ont une portée locale.

Tableaux, parcours avec foreach et tris

Ouvrir cette partie sur STI v2.0 → | QR de lecture en bas de page

Tableaux indexés et associatifs

```
<?php
$notes = [12, 15, 9];
$notes[] = 16;
$ventes = ["lundi" => 7, "mardi" => 5, "jeudi" => 9];
echo $notes[0];          // 12
echo count($notes);      // 4
echo $ventes["mardi"];  // 5
```

array(...) et [...] sont deux syntaxes de construction. Les clés peuvent être des entiers ou des chaînes. Après $tab[10] = "valeur", un ajout par $tab[] utilise normalement l’indice entier suivant, 11, si aucun indice supérieur n’existe.

Parcours recommandé

```
<?php
foreach ($notes as $note) {
    echo $note . "<br>";
}
foreach ($ventes as $jour => $quantite) {
    echo htmlspecialchars($jour, ENT_QUOTES, "UTF-8")
       . " : " . $quantite . " unités<br>";
}
```

foreach fournit directement la valeur, ou la clé et la valeur, sans devoir gérer le pointeur interne du tableau. Le cours n’utilise pas l’ancienne fonction de parcours supprimée dans les versions modernes de PHP.

Tris

• sort($tab) : valeurs croissantes, avec réindexation des clés numériques.

• rsort($tab) : valeurs décroissantes, avec réindexation.

• asort($tab) et arsort($tab) : tri par valeurs, en conservant les clés.

• ksort($tab) et krsort($tab) : tri par clés.

```
<?php
sort($notes, SORT_NUMERIC);
asort($ventes, SORT_NUMERIC);
foreach ($ventes as $jour => $quantite) {
    echo $jour . " : " . $quantite . "<br>";
}
```

Tableaux à plusieurs dimensions

$eleves = [["nom" => "Amal", "note" => 16], ["nom" => "Ali", "note" => 14]];

On accède à la première note avec $eleves[0]["note"]. Un foreach sur $eleves donne chaque ligne sous forme de tableau associatif.

Chaînes et fonctions usuelles

Ouvrir cette partie sur STI v2.0 → | QR de lecture en bas de page

Fonctions essentielles

• trim($texte) : enlève les espaces et certains caractères de contrôle aux extrémités.

• strlen($texte) : longueur en octets ; mb_strlen($texte, "UTF-8") : longueur en caractères avec mbstring.

• substr($texte, $debut, $longueur) : extrait une sous-chaîne en octets.

• strpos($texte, $mot) : position de la première occurrence, ou false.

• str_replace($cherche, $remplace, $texte) : remplacement des occurrences.

• strtoupper et strtolower : conversions de casse pour les caractères ASCII ; pour l'Unicode, utiliser mb_strtoupper et mb_strtolower.

• strcmp($a, $b) : comparaison ; zéro signifie égalité, un nombre négatif ou positif indique l'ordre.

• strrev($texte) : inversion des octets, inadaptée à une chaîne UTF-8 générale.

Exemples

```
<?php
$texte = trim("  Bonjour Tunis  ");
echo substr($texte, 0, 7); // Bonjour
$position = strpos($texte, "Bonjour");
if ($position !== false) {
    echo "Trouvé à la position " . $position; // 0
}
echo str_replace("Tunis", "Sfax", $texte);
echo strtoupper("php"); // PHP
```

Ne pas tester simplement if ($position) : une occurrence située à l'indice 0 serait confondue avec false. La comparaison stricte est nécessaire.

Passer d'une chaîne à un tableau

```
<?php
$langages = explode(";", "HTML;CSS;JavaScript;PHP");
echo count($langages); // 4
$ligne = implode(" / ", $langages);
echo htmlspecialchars($ligne, ENT_QUOTES, "UTF-8");
```

explode découpe une chaîne selon un séparateur non vide. implode assemble les éléments d'un tableau avec un séparateur. Utiliser l'ordre implode($separateur, $tableau).

Application

Lire un texte de formulaire, supprimer ses espaces extérieurs, repérer un mot et afficher le résultat en sécurité. Distinguer un résultat false d'une position égale à zéro.

Dates et heures : timestamp et checkdate

Ouvrir cette partie sur STI v2.0 → | QR de lecture en bas de page

Pourquoi manipuler les dates ?

Un site dynamique peut enregistrer la date système, afficher une date dans un autre format, ajouter une période, comparer des échéances ou traiter des champs DATE et DATETIME.

Un timestamp Unix est un entier représentant un nombre de secondes depuis le 1er janvier 1970 à 00:00:00 UTC. Il désigne un instant ; le fuseau horaire intervient lorsqu'on le transforme en date locale. PHP utilise le type int, dont la capacité dépend de la plateforme. Préférer un environnement 64 bits pour une large plage de dates.

Configurer le fuseau horaire

```
<?php
date_default_timezone_set("Africa/Tunis");
```

Placer cette instruction au début des scripts de dates de ce chapitre, ou définir le même fuseau dans la configuration PHP. Le fuseau horaire MySQL se configure séparément.

1. La fonction checkdate

Signature : checkdate(int $month, int $day, int $year): bool

Elle renvoie true si la date est valide, sinon false. L'ordre est mois, jour, année, et non l'ordre français jour, mois, année.

• Année : de 1 à 32767 inclus.

• Mois : de 1 à 12 inclus.

• Jour : doit exister dans le mois et l'année considérés ; les années bissextiles sont prises en compte.

```
<?php
$j = 30;
$m = 11;
$a = 2026;
if (checkdate($m, $j, $a)) {
    $d = sprintf("%02d/%02d/%04d", $j, $m, $a);
} else {
    $d = "Données invalides";
}
echo $d; // 30/11/2026
```

Tests complémentaires

checkdate(2, 29, 2026) renvoie false ; checkdate(2, 29, 2028) renvoie true. Cette fonction vérifie une date civile ; elle ne crée pas de timestamp et ne valide pas à elle seule une heure.

Application

Contrôler une date de naissance saisie en trois champs. Après la validité civile, vérifier aussi les règles de gestion : par exemple, la date ne doit pas être future.

Dates et heures : date et getdate

Ouvrir cette partie sur STI v2.0 → | QR de lecture en bas de page

2. La fonction date

Signature : date(string $format, ?int $timestamp = null): string

Elle retourne une chaîne formatée. Si le timestamp est omis ou null, elle utilise l'instant courant. L'affichage dépend du fuseau horaire configuré.

```
<?php
date_default_timezone_set("Africa/Tunis");
$d = date("Y-m-d H:i:s");
echo "Nous sommes le : " . $d;
```

Y : année à quatre chiffres ; m : mois à deux chiffres ; d : jour à deux chiffres ; H : heure sur 24 heures ; i : minutes ; s : secondes. Attention : m représente le mois, pas les minutes.

3. La fonction getdate

Signature : getdate(?int $timestamp = null): array

Elle retourne un tableau associatif décrivant la date et l'heure locales, courantes si le paramètre est omis.

```
<?php
date_default_timezone_set("Africa/Tunis");
$a = getdate();
echo "Nous sommes le : " . date("d-m-Y H:i:s") . "<br>";
$libelles = [
    "seconds" => "Secondes", "minutes" => "Minutes",
    "hours" => "Heures", "mday" => "Jour du mois",
    "wday" => "Jour de la semaine", "mon" => "Mois",
    "year" => "Année", "yday" => "Jour de l'année",
    "weekday" => "Nom du jour", "month" => "Nom du mois"
];
foreach ($libelles as $cle => $libelle) {
    echo $libelle . " : " . $a[$cle] . "<br>";
}
```

Contenu du tableau

seconds : 0–59 ; minutes : 0–59 ; hours : 0–23 ; mday : 1–31 ; wday : 0 (dimanche) à 6 ; mon : 1–12 ; year : année ; yday : 0–365. Les clés weekday et month contiennent des noms anglais. La clé numérique 0 contient le timestamp.

Vigilance

La clé est weekday, tout en minuscules. Le numéro de mois de getdate commence à 1, contrairement à getMonth() en JavaScript, qui commence à 0. Pour une présentation localisée en français, envisager IntlDateFormatter si l'extension intl est disponible.

Dates et heures : time et mktime

Ouvrir cette partie sur STI v2.0 → | QR de lecture en bas de page

4. La fonction time

Signature : time(): int

Elle retourne le timestamp Unix courant, en secondes.

```
<?php
date_default_timezone_set("Africa/Tunis");
$a = time();
$b = date("d/m/Y H:i:s", $a);
echo $a . " = " . $b;
```

Le nombre renvoyé évolue à chaque seconde. Il ne faut pas présenter une valeur fixe comme résultat permanent de cet exemple.

5. La fonction mktime

Signature PHP 8 : mktime(int $hour, ?int $minute = null, ?int $second = null, ?int $month = null, ?int $day = null, ?int $year = null): int|false

Elle construit un timestamp à partir de composantes interprétées dans le fuseau local. L'heure est obligatoire en PHP 8 ; les autres paramètres omis ou null prennent les valeurs de la date courante. Fournir tous les paramètres pour un exemple reproductible.

```
<?php
date_default_timezone_set("Africa/Tunis");
$ns = mktime(12, 30, 0, 10, 21, 2026);
if ($ns === false) {
    exit("Date non représentable.");
}
echo "Timestamp : " . $ns . "<br>";
echo "Date : " . date("d-m-Y H:i:s", $ns);
// Date : 21-10-2026 12:30:00
```

Normalisation des dépassements

```
<?php
$suivant = mktime(12, 30, 0, 10, 32, 2026);
if ($suivant !== false) {
    echo date("d-m-Y H:i:s", $suivant);
    // 01-11-2026 12:30:00
}
```

mktime normalise les composantes hors plage ; ce n'est donc pas un validateur. Pour une saisie, vérifier d'abord la date avec checkdate et l'heure avec des bornes appropriées.

Correction importante

L'ancien paramètre is_dst n'existe plus : il a été supprimé en PHP 7. Ne pas transmettre de septième argument. Les règles d'heure d'été éventuelles proviennent du fuseau configuré ; l'ancien texte sur « l'heure d'hiver » était incorrect.

Dates et heures : strtotime et calculs

Ouvrir cette partie sur STI v2.0 → | QR de lecture en bas de page

6. La fonction strtotime

Signature : strtotime(string $datetime, ?int $baseTimestamp = null): int|false

Elle analyse une expression de date en anglais ou certains formats numériques. Les expressions relatives utilisent le timestamp de référence, ou l'instant courant lorsqu'il est omis. Le fuseau par défaut intervient si l'expression n'en précise pas.

```
<?php
date_default_timezone_set("Africa/Tunis");
$base = mktime(12, 0, 0, 10, 21, 2026);
if ($base === false) { exit("Référence invalide."); }
$expressions = [
    "now", "10 September 2026", "+1 day", "+1 week",
    "+1 week 2 days 4 hours 2 seconds"
];
foreach ($expressions as $expression) {
    $ts = strtotime($expression, $base);
    if ($ts === false) {
        echo "Expression invalide<br>";
        continue;
    }
    echo htmlspecialchars($expression, ENT_QUOTES, "UTF-8")
       . " : " . date("Y-m-d H:i:s", $ts)
       . " = " . $ts . " secondes<br>";
}
```

Avec cette référence, +1 day donne 2026-10-22 12:00:00 ; +1 week donne 2026-10-28 12:00:00 ; la dernière expression donne 2026-10-30 16:00:02. « 10 September 2026 » désigne le 10 septembre à minuit.

Ajouter une période à une date

```
<?php
$depart = strtotime("2026-11-30 09:00:00");
if ($depart === false) { exit("Date de départ invalide."); }
$echeance = strtotime("+7 days", $depart);
if ($echeance !== false) {
    echo date("d/m/Y H:i:s", $echeance);
    // 07/12/2026 09:00:00
}
```

Limites

Tester === false : le timestamp 0 est valide. Éviter les dates ambiguës comme 01/02/2026 ; préférer 2026-02-01. strtotime peut normaliser certaines dates civiles impossibles : utiliser checkdate pour une validation stricte. Un mois ajouté en fin de mois peut déborder ; une journée civile ne vaut pas toujours 86400 secondes dans tous les fuseaux.

MySQLi procédural : connexion et lecture

Ouvrir cette partie sur STI v2.0 → | QR de lecture en bas de page

Connexion à MySQL

Activer l’extension mysqli dans l’environnement PHP. Les identifiants ci-dessous sont des exemples à adapter ; utiliser un compte aux droits limités et stocker les secrets hors du dépôt public.

```
<?php
mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);
try {
    $cnx = mysqli_connect(
        "localhost", "utilisateur_cours", "mot_de_passe", "commerce"
    );
    mysqli_set_charset($cnx, "utf8mb4");
    $sql = "SELECT num_art, des_art, pv_art "
         . "FROM articles ORDER BY des_art";
    $res = mysqli_query($cnx, $sql);
    echo "<ul>";
    while ($ligne = mysqli_fetch_assoc($res)) {
        echo "<li>"
           . htmlspecialchars($ligne["des_art"],
                ENT_QUOTES | ENT_SUBSTITUTE, "UTF-8")
           . " : " . number_format((float)$ligne["pv_art"],
                                   3, ",", " ")
           . " DT</li>";
    }
    echo "</ul>";
    mysqli_free_result($res);
    mysqli_close($cnx);
} catch (mysqli_sql_exception $e) {
    error_log($e->getMessage());
    http_response_code(500);
    echo "Une erreur de base de données est survenue.";
}
```

Fonctions utiles

• mysqli_connect ouvre la connexion et peut sélectionner directement la base.

• mysqli_select_db($cnx, $nom) change la base si nécessaire.

• mysqli_query exécute une requête fixe ; pour une entrée utilisateur, préparer la requête.

• mysqli_fetch_assoc lit une ligne sous forme associative ; mysqli_fetch_row fournit des indices numériques.

• mysqli_num_rows compte les lignes d’un résultat de SELECT.

• mysqli_affected_rows indique le nombre de lignes affectées par une modification ; mysqli_insert_id fournit le dernier identifiant AUTO_INCREMENT créé sur la connexion.

• mysqli_free_result libère le résultat ; mysqli_close ferme la connexion.

Les erreurs techniques sont journalisées côté serveur, sans afficher les identifiants ni les détails internes aux visiteurs. Tous les appels de base de données de ce cours sont procéduraux.

MySQLi procédural : requêtes préparées

Ouvrir cette partie sur STI v2.0 → | QR de lecture en bas de page

Paramétrer plutôt que concaténer

Les marqueurs ? représentent des valeurs et empêchent qu'une saisie soit interprétée comme du code SQL. Ils ne remplacent pas les noms de tables ou de colonnes : pour un tri dynamique, utiliser une liste blanche de noms autorisés.

Le code ci-dessous utilise une connexion $cnx ouverte comme dans la fiche précédente, avec le même dispositif de gestion des erreurs.

```
<?php
$codeBrut = $_GET["code"] ?? null;
if (!is_string($codeBrut)) { exit("Code invalide."); }
$code = filter_var($codeBrut, FILTER_VALIDATE_INT, [
    "options" => ["min_range" => 1]
]);
if ($code === false) { exit("Code invalide."); }
$stmt = mysqli_prepare($cnx,
    "SELECT des_art, pv_art FROM articles WHERE num_art = ?"
);
mysqli_stmt_bind_param($stmt, "i", $code);
mysqli_stmt_execute($stmt);
mysqli_stmt_bind_result($stmt, $designation, $prix);
if (mysqli_stmt_fetch($stmt)) {
    echo htmlspecialchars($designation, ENT_QUOTES, "UTF-8");
    echo " : " . number_format((float)$prix, 3, ",", " ");
} else {
    echo "Article introuvable.";
}
mysqli_stmt_close($stmt);
```

Insérer avec des valeurs liées

```
<?php
// Valeurs validées, démonstration indépendante.
$nom = "Site PHP";
$url = "https://www.php.net/";
$stmt = mysqli_prepare($cnx,
    "INSERT INTO sites (nom, url) VALUES (?, ?)"
);
mysqli_stmt_bind_param($stmt, "ss", $nom, $url);
mysqli_stmt_execute($stmt);
echo "Lignes insérées : " . mysqli_stmt_affected_rows($stmt);
mysqli_stmt_close($stmt);
```

Cet INSERT suppose la table sites créée dans la fiche suivante. Dans bind_param, i désigne un entier, d un flottant, s une chaîne et b un contenu binaire envoyé par morceaux. Les variables liées sont passées par référence. Pour un montant DECIMAL, transmettre une chaîne décimale validée peut éviter une conversion flottante inutile.

TP : répertoire de sites en MySQLi procédural

Ouvrir cette partie sur STI v2.0 → | QR de lecture en bas de page

Préparation de la table dans commerce

```
CREATE TABLE sites (
  id INT AUTO_INCREMENT PRIMARY KEY,
  nom VARCHAR(100) NOT NULL,
  url VARCHAR(255) NOT NULL
) ENGINE=InnoDB;
INSERT INTO sites (nom, url)
VALUES ('PHP', 'https://www.php.net/');
```

Affichage corrigé

Le fragment suivant se place dans le body d'une page HTML, avec une connexion $cnx existante à commerce. Les URL sont contrôlées avant d'être utilisées dans href : l'échappement HTML seul ne bloque pas un schéma javascript:.

```
<table>
  <caption>Répertoire de sites</caption>
  <thead><tr><th>Nom du site</th><th>URL</th></tr></thead>
  <tbody>
<?php
$res = mysqli_query($cnx, "SELECT nom, url FROM sites ORDER BY nom");
while ($row = mysqli_fetch_assoc($res)) {
    $nom = htmlspecialchars($row["nom"],
                            ENT_QUOTES | ENT_SUBSTITUTE, "UTF-8");
    $urlBrute = $row["url"];
    $schema = strtolower(parse_url($urlBrute, PHP_URL_SCHEME) ?? "");
    if (filter_var($urlBrute, FILTER_VALIDATE_URL) === false
        || !in_array($schema, ["http", "https"], true)) {
        continue;
    }
    $url = htmlspecialchars($urlBrute,
                            ENT_QUOTES | ENT_SUBSTITUTE, "UTF-8");
    echo '<tr><td><a href="' . $url . '">' . $nom
       . '</a></td><td>' . $url . '</td></tr>';
}
mysqli_free_result($res);
mysqli_close($cnx);
?>
  </tbody>
</table>
```

Travail demandé

1. Ajouter un formulaire nom/url, puis traiter le POST avec validation et requête préparée.

2. Limiter les longueurs en cohérence avec la table et n'accepter que http et https.

3. Ajouter une modification et une suppression paramétrées par id, avec contrôle d'autorisation et jeton CSRF pour les opérations sensibles.

4. Vérifier l'affichage d'un nom contenant une apostrophe et un caractère &.

Fonctions complémentaires et superglobales

Ouvrir cette partie sur STI v2.0 → | QR de lecture en bas de page

Variables et superglobales

$_GET et $_POST contiennent les paramètres ; $_SERVER des informations sur la requête et le serveur ; $_FILES les fichiers reçus. Une superglobale reste accessible dans une fonction, mais ses données externes ne sont pas automatiquement fiables.

```
<?php
$valeur = $_POST["nom"] ?? null;
if (!is_string($valeur)) { exit("Nom invalide."); }
$nom = trim($valeur);
if ($nom === "") { exit("Nom obligatoire."); }
echo htmlspecialchars($nom, ENT_QUOTES, "UTF-8");
```

empty($x) vaut notamment true pour 0, "0", "", null et une variable absente. Ne pas l’utiliser pour rejeter une saisie numérique si zéro est autorisé. isset ne distingue pas une valeur null d’une clé absente ; array_key_exists le permet.

Fonctions numériques

```
<?php
echo abs(-7);          // 7
echo floor(4.8);       // 4
echo ceil(4.2);        // 5
echo round(12.345, 2); // 12.35
echo sqrt(81);        // 9
echo pow(2, 3);       // 8
echo min(3, 9, 2);    // 2
echo max(3, 9, 2);    // 9
```

Afficher ou renvoyer

```
<?php
function prixTTC($prixHT, $taux) {
    return $prixHT * (1 + $taux);
}
$total = prixTTC(100, 0.19);
echo number_format($total, 3, ",", " ") . " DT";
```

echo est une construction du langage, pas une fonction mathématique ; elle produit une sortie et ne renvoie pas de valeur exploitable. return fournit un résultat à l’appelant. number_format crée une chaîne d’affichage et ne change pas la valeur de calcul.

Application

Contrôler une note égale à zéro sans la confondre avec une valeur absente. Prévoir séparément la validation, le calcul et l’affichage.

MySQLi : types de résultats et fonctions fetch

Ouvrir cette partie sur STI v2.0 → | QR de lecture en bas de page

mysqli_query : SELECT ou modification

Pour une requête SELECT réussie, mysqli_query retourne un objet mysqli_result. Pour INSERT, UPDATE ou DELETE réussi, elle retourne true, pas le nombre de lignes. En mode non strict, une erreur peut produire false ; avec le mode strict du cours, une erreur SQL déclenche une exception.

```
<?php
// $cnx : connexion ouverte à commerce, mode strict activé.
$res = mysqli_query($cnx,
    "SELECT num_art, des_art FROM articles ORDER BY num_art"
);
echo "Lignes du résultat : " . mysqli_num_rows($res);
while ($ligne = mysqli_fetch_array($res, MYSQLI_BOTH)) {
    // Deux accès à la même colonne :
    echo $ligne[0];              // num_art
    echo $ligne["num_art"];     // même valeur
}
mysqli_free_result($res);
```

Choisir le mode de lecture

• mysqli_fetch_assoc : clés portant les noms des colonnes.

• mysqli_fetch_row : indices numériques 0, 1, 2…

• mysqli_fetch_array : MYSQLI_ASSOC, MYSQLI_NUM ou MYSQLI_BOTH, ce dernier étant le mode par défaut.

Ces fonctions lisent une seule ligne et font avancer le curseur. Elles renvoient null lorsqu’il n’y a plus de ligne, et peuvent renvoyer false en cas d’échec. Ne pas appeler deux fonctions fetch successivement en pensant relire la même ligne.

Compter efficacement

```
<?php
$res = mysqli_query($cnx, "SELECT COUNT(*) AS nb FROM articles");
$ligne = mysqli_fetch_assoc($res);
echo $ligne["nb"];
// mysqli_num_rows($res) vaut 1 : une ligne portant le compteur.
mysqli_free_result($res);
```

Après un UPDATE

mysqli_affected_rows indique normalement les lignes réellement modifiées ; zéro peut signifier aucune correspondance ou une valeur déjà identique. Un succès d’exécution ne prouve pas qu’une valeur a changé. mysqli_insert_id fournit le dernier identifiant AUTO_INCREMENT de la connexion ; son type peut être int ou string selon la taille.

Application

Lire le même résultat avec chacun des trois modes, en relançant la requête à chaque fois. Expliquer pourquoi COUNT(*) et mysqli_num_rows ne donnent pas toujours le même nombre.

Projet 1 : gestion des élèves — énoncé et base

Ouvrir cette partie sur STI v2.0 → | QR de lecture en bas de page

Projet adapté de la plateforme

Construire un site « Gestion des élèves — Lycée Rafèha ». La page index.html contient un en-tête, une navigation verticale, un iframe nommé cadre et un pied de page. Les pages d'inscription, de liste, de modification et de statistiques s'affichent dans le cadre.

Base ecole

```
CREATE DATABASE ecole CHARACTER SET utf8mb4;
USE ecole;
CREATE TABLE eleves (
  Id VARCHAR(10) PRIMARY KEY,
  Nom_prenom VARCHAR(60) NOT NULL,
  Moyenne DECIMAL(4,2) NOT NULL CHECK (Moyenne BETWEEN 0 AND 20),
  DateNaissance DATE NOT NULL
);
INSERT INTO eleves VALUES
('A01', 'Benali Amine', 17.50, '2008-03-14'),
('A02', 'Trabelsi Sara', 16.25, '2008-07-02'),
('B01', 'Gharbi Yassine', 14.75, '2007-11-25'),
('C01', 'Ayadi Nour', 10.50, '2008-05-30'),
('D01', 'Cherif Ines', 7.25, '2008-09-17');
```

L'Id commence par une majuscule qui indique sa catégorie. Le nom complet comporte deux à cinq mots de lettres. La moyenne est comprise entre 0 et 20. La date de naissance doit être passée.

Fichiers à produire

index.html, inscription.html, style.css, validation.js, connexion.php, ajouter.php, liste.php, modifier.php et statistiques.php.

Consignes HTML et CSS

Navigation verticale et mise en page avec float, marge latérale et clear. Ne pas employer flex dans ce TP. Les liens ciblent cadre ; l'iframe a un title. Les champs portent label, id et name. Ajouter des transitions sur les liens, boutons et champs au focus.

Consignes JavaScript et PHP

Utiliser let en JavaScript. Nettoyer les espaces, contrôler les champs et afficher les erreurs avec textContent. Refaire tous les contrôles en PHP. Employer les requêtes préparées de MySQLi procédural pour les valeurs reçues. La normalisation en majuscules d'un identifiant ne remplace pas sa validation.

Évaluation proposée sur 20

HTML et navigation : 4 ; CSS et transitions : 4 ; validation JavaScript : 4 ; PHP et données : 6 ; tests et lisibilité : 2. Cette répartition est proposée pour cette édition.

Projet 1 : formulaire et validation JavaScript

Ouvrir cette partie sur STI v2.0 → | QR de lecture en bas de page

inscription.html : contenu du formulaire

```
<form id="inscription" method="post" action="ajouter.php">
  <label for="Id">Identifiant</label>
  <input id="Id" name="Id" maxlength="10" required>
  <label for="Nom_prenom">Nom complet</label>
  <input id="Nom_prenom" name="Nom_prenom" maxlength="60" required>
  <label for="Moyenne">Moyenne</label>
  <input id="Moyenne" name="Moyenne" type="number"
         min="0" max="20" step="0.01" required>
  <label for="DateNaissance">Date de naissance</label>
  <input id="DateNaissance" name="DateNaissance" type="date" required>
  <button type="submit">Ajouter</button>
  <button type="reset">Effacer</button>
</form>
<p id="erreurs" role="alert"></p>
<script src="validation.js" defer></script>
```

validation.js

Réutiliser dateNonFuture définie dans la fiche « Médias, styles et dates de formulaire ». Ajouter le contrôle strictement passé indiqué ci-dessous.

```
let formulaire = document.getElementById("inscription");
formulaire.addEventListener("submit", function (event) {
  let id = document.getElementById("Id");
  let nom = document.getElementById("Nom_prenom");
  let champNote = document.getElementById("Moyenne");
  let date = document.getElementById("DateNaissance").value;
  id.value = id.value.trim().toUpperCase();
  nom.value = nom.value.trim().replace(/\s+/g, " ");
  let mots = nom.value.split(" ");
  let note = Number(champNote.value);
  let jour = new Date();
  let aujourdhui = jour.getFullYear() + "-"
    + String(jour.getMonth() + 1).padStart(2, "0") + "-"
    + String(jour.getDate()).padStart(2, "0");
  let valide = /^[A-Z][A-Z0-9]{0,9}$/.test(id.value)
    && /^[\p{L} ]+$/u.test(nom.value)
    && mots.length >= 2 && mots.length <= 5
    && champNote.value !== "" && Number.isFinite(note)
    && note >= 0 && note <= 20 && dateNonFuture(date)
    && date < aujourdhui;
  if (!valide) {
    event.preventDefault();
    document.getElementById("erreurs").textContent =
      "Vérifiez l'identifiant, le nom, la moyenne et la date.";
  }
});
```

La comparaison à aujourdhui exclut la date du jour, après validation civile. Ne pas utiliser toISOString pour construire cette date locale : la date UTC peut être différente.

Projet 1 : connexion et insertion PHP

Ouvrir cette partie sur STI v2.0 → | QR de lecture en bas de page

connexion.php — environnement pédagogique local

```
<?php
mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);
$cnx = mysqli_connect("localhost", "utilisateur_cours",
                      "mot_de_passe", "ecole");
mysqli_set_charset($cnx, "utf8mb4");
date_default_timezone_set("Africa/Tunis");
```

Adapter les identifiants. Inclure ce fichier avec require __DIR__ . "/connexion.php". La gestion des exceptions présentée au cours doit entourer les opérations ; ne pas afficher les détails SQL au visiteur.

ajouter.php — traitement principal

```
<?php
require __DIR__ . "/connexion.php";
if ($_SERVER["REQUEST_METHOD"] !== "POST") { exit("POST requis."); }
foreach (["Id", "Nom_prenom", "Moyenne", "DateNaissance"] as $cle) {
    if (!isset($_POST[$cle]) || !is_string($_POST[$cle])) {
        exit("Données invalides.");
    }
}
$id = strtoupper(trim($_POST["Id"]));
$nom = preg_replace('/\s+/u', ' ', trim($_POST["Nom_prenom"]));
$moy = filter_var($_POST["Moyenne"], FILTER_VALIDATE_FLOAT);
$date = $_POST["DateNaissance"];
if (!preg_match('/^[A-Z][A-Z0-9]{0,9}$/D', $id)
    || $nom === null || !preg_match('/^\p{L}+(?: \p{L}+){1,4}$/uD', $nom)
    || strlen($nom) > 60 || $moy === false || $moy < 0 || $moy > 20
    || !preg_match('/^(\d{4})-(\d{2})-(\d{2})$/D', $date, $p)
    || !checkdate((int)$p[2], (int)$p[3], (int)$p[1])
    || $date >= date("Y-m-d")) { exit("Valeurs invalides."); }
$stmt = mysqli_prepare($cnx,
    "INSERT INTO eleves (Id, Nom_prenom, Moyenne, DateNaissance) "
    . "VALUES (?, ?, ?, ?)"
);
mysqli_stmt_bind_param($stmt, "ssds", $id, $nom, $moy, $date);
mysqli_stmt_execute($stmt);
echo "Élève ajouté.";
mysqli_stmt_close($stmt);
mysqli_close($cnx);
```

La limite strlen est ici volontairement prudente en octets ; l’adapter avec mb_strlen pour exactement 60 caractères Unicode. Gérer un Id déjà présent par une erreur métier compréhensible. Pour une mise en ligne, ajouter authentification, autorisations et jeton CSRF.

Projet 1 : liste, modification et statistiques

Ouvrir cette partie sur STI v2.0 → | QR de lecture en bas de page

liste.php : lecture et affichage

```
<?php
require __DIR__ . "/connexion.php";
$res = mysqli_query($cnx,
    "SELECT Id, Nom_prenom, Moyenne, DateNaissance "
    . "FROM eleves ORDER BY Id"
);
echo "<table><tr><th>Id</th><th>Nom complet</th><th>Moyenne</th></tr>";
while ($e = mysqli_fetch_assoc($res)) {
    echo "<tr>";
    foreach (["Id", "Nom_prenom", "Moyenne"] as $col) {
        echo "<td>" . htmlspecialchars((string)$e[$col],
                        ENT_QUOTES | ENT_SUBSTITUTE, "UTF-8") . "</td>";
    }
    echo "</tr>";
}
echo "</table>";
mysqli_free_result($res);
mysqli_close($cnx);
```

modifier.php : principe de correction

Un formulaire POST envoie l'Id et la nouvelle moyenne. Après vérification des types, du format de l'Id et de la moyenne entre 0 et 20, utiliser le fragment ci-dessous avec une connexion ouverte :

```
<?php
$stmt = mysqli_prepare($cnx,
    "UPDATE eleves SET Moyenne = ? WHERE Id = ?"
);
mysqli_stmt_bind_param($stmt, "ds", $moyenneValidee, $idValide);
mysqli_stmt_execute($stmt);
echo mysqli_stmt_affected_rows($stmt) . " ligne(s) modifiée(s).";
mysqli_stmt_close($stmt);
```

statistiques.php : requêtes à exploiter

```
SELECT COUNT(*) AS effectif, AVG(Moyenne) AS moyenne_classe,
       MIN(Moyenne) AS minimum, MAX(Moyenne) AS maximum FROM eleves;
SELECT LEFT(Id, 1) AS categorie, COUNT(*) AS effectif,
       AVG(Moyenne) AS moyenne FROM eleves
GROUP BY LEFT(Id, 1) ORDER BY categorie;
SELECT Id, Nom_prenom,
       TIMESTAMPDIFF(YEAR, DateNaissance, CURRENT_DATE()) AS age
FROM eleves ORDER BY Nom_prenom;
```

Compléter l'affichage HTML, les anniversaires du jour et les cas de table vide. Les fragments constituent des pistes de correction à assembler avec les contrôles du cours, pas un service prêt à déployer publiquement.

Projet 2 : recharge électrique — énoncé et base

Ouvrir cette partie sur STI v2.0 → | QR de lecture en bas de page

Contexte adapté de la plateforme

Gérer les recharges d'une station. Construire la même navigation à iframe que dans le projet précédent, avec float et transitions. Chaque recharge possède Id, Nom_prenom, Borne, Kwh, DateRecharge et HeureRecharge.

La première lettre de l'Id indique le type : L lente, N normale, R rapide, U ultra-rapide. L'énergie est comprise entre 1 et 100 kWh. La date ne doit pas être future ; l'heure doit appartenir aux plages 08:00–12:00 ou 13:00–18:00.

Règle simplifiée retenue pour cette édition

Une borne ne peut recevoir qu'un enregistrement pour une même date et une même heure de départ. Ce modèle ne gère pas encore une durée de recharge ni les chevauchements d'intervalles : ce serait une extension du projet.

```
CREATE DATABASE station CHARACTER SET utf8mb4;
USE station;
CREATE TABLE recharges (
  Id VARCHAR(10) PRIMARY KEY,
  Nom_prenom VARCHAR(60) NOT NULL,
  Borne VARCHAR(10) NOT NULL,
  Kwh DECIMAL(5,2) NOT NULL CHECK (Kwh BETWEEN 1 AND 100),
  DateRecharge DATE NOT NULL,
  HeureRecharge TIME NOT NULL,
  CONSTRAINT uq_depart UNIQUE (Borne, DateRecharge, HeureRecharge),
  CONSTRAINT ck_horaire CHECK (
    HeureRecharge BETWEEN '08:00:00' AND '12:00:00'
    OR HeureRecharge BETWEEN '13:00:00' AND '18:00:00'
  )
);
INSERT INTO recharges VALUES
('L01', 'Benali Amine', 'B01', 20.00, '2026-10-20', '09:00:00'),
('R01', 'Trabelsi Sara', 'B02', 35.50, '2026-10-20', '14:00:00');
```

Livrables

index.html, inscription.html, style.css, validation.js et quatre pages PHP : ajouter, liste, modifier et statistiques. Adapter connexion.php pour utiliser station.

Contrôles et affichage

Nettoyer le nom, contrôler l'Id et l'énergie, vérifier date et heure en JavaScript puis PHP. Afficher une horloge et changer la couleur de fond parmi trois couleurs prédéfinies. Fournir un tableau de l'activité du jour, sans présenter une borne comme « libre maintenant » sur la seule base d'une heure de départ.

Projet 2 : horaires et insertion paramétrée

Ouvrir cette partie sur STI v2.0 → | QR de lecture en bas de page

Validation JavaScript d’un horaire

```
function horaireValide(heure) {
  if (!/^\d{2}:\d{2}$/.test(heure)) { return false; }
  let morceaux = heure.split(":").map(Number);
  let h = morceaux[0], m = morceaux[1];
  if (h > 23 || m > 59) { return false; }
  let minutes = h * 60 + m;
  return (minutes >= 480 && minutes <= 720)
      || (minutes >= 780 && minutes <= 1080);
}
```

Cette édition inclut les bornes 12:00 et 18:00 comme heures de départ autorisées. Modifier explicitement la règle si une durée minimale doit être terminée avant la fermeture.

Validation PHP : heure HH:MM reçue du formulaire

```
<?php
$heure = $_POST["HeureRecharge"] ?? null;
if (!is_string($heure)
    || !preg_match('/^(\d{2}):(\d{2})$/D', $heure, $p)) {
    exit("Heure invalide.");
}
$h = (int)$p[1]; $m = (int)$p[2];
$minutes = $h * 60 + $m;
if ($h > 23 || $m > 59 || !(($minutes >= 480 && $minutes <= 720)
    || ($minutes >= 780 && $minutes <= 1080))) {
    exit("Hors horaires.");
}
$heureSQL = $heure . ":00";
```

Insertion après tous les autres contrôles

```
<?php
// $cnx ouverte sur station ; toutes les valeurs ci-dessous sont validées.
$stmt = mysqli_prepare($cnx,
  "INSERT INTO recharges "
  . "(Id, Nom_prenom, Borne, Kwh, DateRecharge, HeureRecharge) "
  . "VALUES (?, ?, ?, ?, ?, ?)"
);
mysqli_stmt_bind_param($stmt, "sssdss", $id, $nom, $borne,
                       $kwh, $dateRecharge, $heureSQL);
mysqli_stmt_execute($stmt);
mysqli_stmt_close($stmt);
```

Vérifier notamment l’Id avec /^[LNRU][A-Z0-9]{1,9}$/ et la date avec checkdate. La contrainte UNIQUE tranche un conflit concurrent même si un SELECT préalable semblait autoriser la place. Traiter les doublons avec un message adapté sans divulguer les détails SQL.

Projet 2 : statistiques, coûts et tests

Ouvrir cette partie sur STI v2.0 → | QR de lecture en bas de page

Statistiques SQL

```
SELECT LEFT(Id, 1) AS type_recharge,
       COUNT(*) AS nombre, SUM(Kwh) AS energie
FROM recharges GROUP BY LEFT(Id, 1) ORDER BY type_recharge;
SELECT Borne, HeureRecharge, Id, Kwh
FROM recharges WHERE DateRecharge = CURRENT_DATE()
ORDER BY Borne, HeureRecharge;
```

Le second résultat est un planning des départs enregistrés, pas une preuve de disponibilité en temps réel.

Calcul d’un coût estimé en PHP

Les tarifs suivants sont inventés pour l’exercice et exprimés en DT/kWh ; ils ne représentent pas une grille commerciale réelle.

```
<?php
$tarifs = ["L" => 0.400, "N" => 0.500, "R" => 0.650, "U" => 0.800];
$res = mysqli_query($cnx,
    "SELECT Id, Nom_prenom, Kwh FROM recharges ORDER BY Id"
);
while ($ligne = mysqli_fetch_assoc($res)) {
    $type = substr($ligne["Id"], 0, 1);
    if (!isset($tarifs[$type])) { continue; }
    $cout = (float)$ligne["Kwh"] * $tarifs[$type];
    echo htmlspecialchars($ligne["Id"], ENT_QUOTES, "UTF-8")
       . " : " . number_format($cout, 3, ",", " ") . " DT<br>";
}
mysqli_free_result($res);
```

Pour une facturation réelle, enregistrer le tarif appliqué, fixer une règle d’arrondi et employer des calculs décimaux adaptés. Ne pas recalculer un ancien montant avec un tarif qui a changé.

Jeu de tests obligatoire

- Énergie : 0, 1, 100, 101 ; nom vide ou contenant un chiffre.
- Horaires : 07:59, 08:00, 12:00, 12:01, 13:00, 18:00, 18:01.
- Date future et date civile impossible ; doublon d’Id ; doublon de départ sur la même borne.
- Nom contenant une apostrophe ou & ; base vide ; serveur indisponible.
- Validation contournée : soumettre directement une requête POST invalide et vérifier le refus PHP.

Travail complémentaire

Ajouter l’heure de fin et empêcher les chevauchements pour une borne. Définir une stratégie transactionnelle avant de prétendre gérer les réservations simultanées. Prévoir autorisations et jeton CSRF pour toute modification sensible.

Synthèse et contrôle des acquis

Ouvrir cette partie sur STI v2.0 → | QR de lecture en bas de page

Parcours d’un projet Web

1. Définir les données et les règles de gestion.
2. Concevoir le schéma relationnel et ses contraintes.
3. Construire une page HTML sémantique et des formulaires accessibles.
4. Appliquer les styles CSS, contrôler le modèle de boîte et la lisibilité.
5. Ajouter JavaScript avec let, des fonctions et des écouteurs d’événements.
6. Valider les données côté PHP, utiliser MySQLi procédural et des requêtes préparées.
7. Échapper les sorties selon leur contexte et tester les erreurs.

Vérification technique

• HTML : id uniques, name sur les champs à transmettre, label correctement associés.

• CSS : unités sans espace, bordures incluses dans le calcul des dimensions.

• JavaScript : déclarations avant utilisation, portée de bloc, gestion de null et de NaN.

• SQL : schéma sélectionné, noms de colonnes cohérents, clés étrangères explicites, WHERE vérifié avant UPDATE/DELETE.

• PHP : balise <?php, tableaux parcourus avec foreach, aucune ancienne API de connexion, contrôle des valeurs reçues.

• Dates : fuseau configuré, formats Y-m-d et H:i:s, timestamps en secondes en PHP et en millisecondes en JavaScript.

Exercices de synthèse

A. Réaliser un formulaire d’inscription d’élèves et afficher les données reçues sans injection HTML.

B. Afficher les locations d’un client pour une période, avec une requête préparée.

C. Calculer une échéance à sept jours puis vérifier que l’affichage est au format français.

D. Dans commerce, lister les articles jamais vendus et les clients dont le total dépasse 1000 DT.

Repères documentaires officiels

HTML et CSS : https://developer.mozilla.org/fr/

JavaScript : https://developer.mozilla.org/fr/docs/Web/JavaScript

PHP et MySQLi : https://www.php.net/manual/fr/

MySQL 8.0 : https://dev.mysql.com/doc/refman/8.0/en/

Révision : questions et réponses essentielles

Ouvrir cette partie sur STI v2.0 → | QR de lecture en bas de page

Questions

1. Quelle différence entre id, class et name en HTML ?

2. Un style interne est-il toujours prioritaire sur un style externe ?

3. Quelle place conserve un élément positionné en relative ?

4. Pourquoi i n’est-il plus accessible après for (let i = 0; ...) ?

5. Pourquoi tester prompt contre null avant conversion ?

6. Pourquoi COUNT(*) et COUNT(colonne) peuvent-ils différer ?

7. Où placer un filtre portant sur un agrégat de groupes ?

8. Pourquoi une requête SELECT COUNT(*) retourne-t-elle une ligne ?

9. checkdate renvoie-t-elle un timestamp ?

10. Comment protéger les valeurs SQL et les sorties HTML ?

Réponses

1. id identifie un élément unique ; class sert à partager des sélecteurs ; name est notamment la clé de transmission d’un champ de formulaire.

2. Non. L’emplacement interne/externe ne suffit pas ; considérer origine, importance, couches, spécificité et ordre d’apparition.

3. Sa place initiale reste réservée, même si son rendu est décalé.

4. let déclare la variable dans la portée de la boucle, et non à l’extérieur.

5. Annuler renvoie null ; Number(null) vaut 0 et pourrait transformer une annulation en saisie numérique valide.

6. COUNT(colonne) ignore NULL ; COUNT(*) compte les lignes.

7. Dans HAVING après GROUP BY. WHERE filtre les lignes avant regroupement.

8. Sans GROUP BY, l’agrégation globale produit une ligne contenant le compteur, même si la table est vide ; mysqli_num_rows vaut alors 1.

9. Non : elle renvoie un booléen et vérifie une date civile dans l’ordre mois, jour, année.

10. Requêtes préparées pour les valeurs SQL ; htmlspecialchars pour le texte HTML et les attributs ordinaires entre guillemets. Les contextes URL, JavaScript et CSS exigent aussi des règles propres.

Auto-évaluation

Pour chaque réponse, produire un exemple de cinq à dix lignes. Tester un cas normal, un cas limite et un cas invalide. Les tests de sécurité ne doivent être réalisés que sur votre propre environnement pédagogique.

Sources, périmètre et conventions de cette édition

Ouvrir cette partie sur STI v2.0 → | QR de lecture en bas de page

Supports utilisés

• PDF original fourni : Cours html5-js-PHP-BD -.pdf, 65 pages.

• Plateforme STI d'Aymen Essouyah : https://aymenessouyah.github.io/STiV2.0/

• Pages de cours consultées : cours/html5.html, cours/css3.html, cours/javascript.html, cours/sql.html, cours/sql-bases-ldd-lmd-lcd.html, cours/php.html et cours/php-mysqli.html.

Édition Smart Book

Cette édition connectée associe des liens et QR codes aux fiches. Un filigrane QR très pâle renvoie à la plateforme ; il sert de marquage, pas de code de lecture fiable ni de protection contre la copie. Le code contrasté en bas de page est celui à scanner. Cette édition fusionne la première réorganisation du PDF avec les compléments des pages de cours de la plateforme. Les explications et exemples sont reformulés pour une lecture imprimée. Elle ne reproduit pas toutes les animations, vidéos, fenêtres interactives, quiz et corrections téléchargeables. Les deux projets de synthèse sont des adaptations pédagogiques avec fragments de correction ; les tarifs et certaines règles simplifiées y sont signalés comme choix de cette édition.

Conventions techniques

• JavaScript : let pour les variables et gestion explicite de leur portée.

• PHP : syntaxe PHP 8, tableaux parcourus avec foreach ou boucles classiques, sans ancienne fonction de parcours supprimée.

• Accès aux bases : uniquement MySQLi procédural, appels mysqli_* ; aucune API mysql_* ni accès orienté objet ou PDO.

• SQL : exemples MySQL 8.0.16 ou ultérieur ; contraintes référentielles explicites et attention au moteur et à la version.

• Dates : exemples actualisés pour 2026 ; 2028 pour illustrer une année bissextile ; fuseau PHP Africa/Tunis.

Documentation de référence

HTML, CSS et JavaScript : https://developer.mozilla.org/fr/

PHP : https://www.php.net/manual/fr/

MySQL : https://dev.mysql.com/doc/refman/8.0/en/

Mise en œuvre

Les schémas gestion_location, commerce, ecole et station sont des domaines distincts. Sélectionner la bonne base avant chaque activité. Certains fragments supposent des variables validées ou une connexion déjà ouverte, explicitement indiquées dans le texte. Les opérations destructives et les changements de structure doivent être testés sur une copie sauvegardée.

Pour une mise en production

Les projets de lycée ne constituent pas une application publique sécurisée complète. Ajouter notamment authentification, autorisations, protection CSRF, configuration des secrets, journalisation et stratégie de concurrence avant un déploiement réel.